

MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation

Huawei Lin^{1,2,*}, Peng Li¹, Jie Song¹, Fuxin Jiang¹, Tieying Zhang^{1,†}

¹ByteDance Inc., ²Rochester Institute of Technology

*Work done during an internship at the ByteBrain team, †Corresponding author

Abstract

Large language model (LLM) agents rely on reusable skills to solve complex tasks, but existing skill creation approaches often treat skills as isolated, static artifacts, limiting reusability, reliability, and long-term improvement. We propose **MUSE-Autoskill Agent (Memory-Utilizing Skill Evolution)**, a skill-centric agent framework that creates, reuses, and refines skills under a unified lifecycle: creation, memory, management, evaluation, and refinement. MUSE creates skills on demand, stores them across tasks, retrieves them through a skill catalog, and accumulates per-skill experience for later reuse and adaptation. Across the main reported settings on SkillsBench and SkillLearnBench, MUSE-Autoskill outperforms Hermes, Codex, and Claude Code. On SkillsBench, its self-created skills surpass human-authored skills on the successfully covered subset (85.24% vs. 81.17%), showing that lifecycle-managed skills can distill agent experience into highly effective reusable assets; MUSE-created skills also transfer to Hermes more effectively than Codex- or Claude-created skills, reaching 51.90% accuracy under transfer. These results highlight the importance of treating skills as long-lived, experience-aware, and testable assets.

Date: July 7, 2026

Correspondence: Tieying Zhang, tieying.zhang@bytedance.com

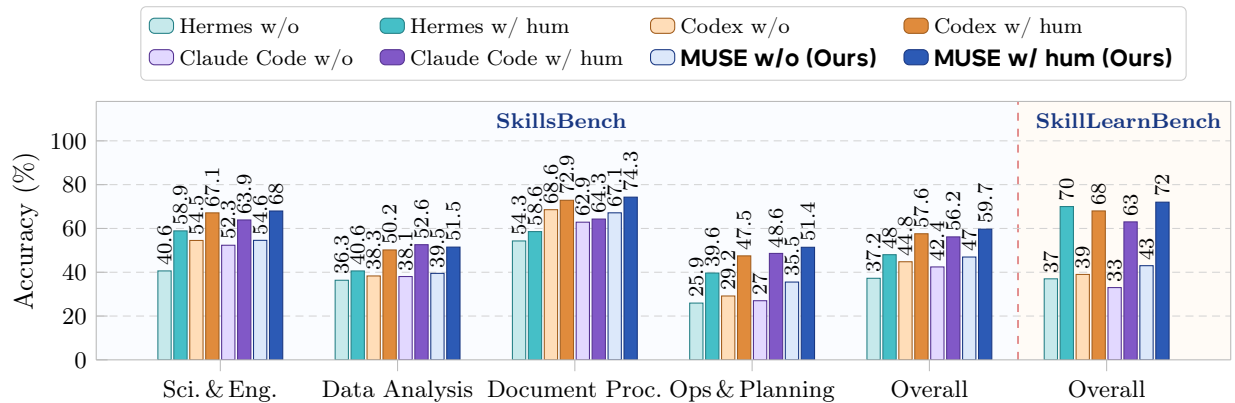


Figure 1 MUSE-Autoskill (ours) leads across SkillsBench and SkillLearnBench. Accuracy (%) of four GPT-5.5-backed agents. The first four groups are the 75-task SkillsBench super-domains; *SkillsBench* is the strict all-task average; *SkillLearnBench* is the 100-instance benchmark average. Paired bars per agent: lighter = *without skills*, saturated = *with human skills*. MUSE-Autoskill achieves the highest human-skill score in 3 of 4 SkillsBench domains, on the SkillsBench average (59.7%), and on SkillLearnBench (72.0%). See Section 4 and Tables 2–3.

1 Introduction

Skills for agents. Large language model (LLM) agents are increasingly tasked with solving complex, real-world problems that involve interacting with external tools, data, and code, often spanning many steps and disparate domains [8, 17, 37]. As task scope grows, raw model reasoning alone is insufficient: agents need access to reusable units of capability, namely skills, that encapsulate procedures, executable code, or domain-specific instructions and can be composed into solutions [2, 28]. Skills are emerging as the natural abstraction for scalable agent systems because they decouple capability from monolithic model weights, enabling modular execution and the accumulation of structured domain knowledge [2, 33]. The central open question is how to enable agents to continuously improve their capabilities through skills they can obtain, organize, and refine on their own, without relying on human authoring at every step.

Limits of AutoSkill. A growing line of work uses LLMs to synthesize skills automatically, starting from Voyager’s executable code library in Minecraft [28] and extending to general-purpose agents via AutoSkill [36], EvoSkill [1], and SkillGen [15]. More recent approaches use reinforcement learning to jointly optimize skill selection, use, and distillation (Skill1 [25]) or to train a dedicated skill curator (SkillOS [18]). On the production side, Anthropic’s Agent Skills [2] standardize skills as portable folders of instructions and scripts. While these methods successfully expand agent functionality, they typically cover only part of the skill lifecycle and leave four practical gaps: (i) a creation–usage mismatch, where skills are produced without access to the agent’s runtime context; (ii) no structured per-skill memory that accumulates free-form experience about individual skills across tasks; (iii) static, unvalidated skills without unit-test-driven evaluation or refinement; and (iv) poor context handling, where flat conversation histories truncate or overflow on long-horizon tasks.

Skill lifecycle. We argue that skills should not be one-off generation outputs but long-lived, evolving assets of an agent system. A useful skill is created on demand within the agent’s reasoning loop, stored with associated experience and metadata [19, 20, 27], retrieved when contextually relevant, validated through tests and runtime feedback, and continuously refined as new evidence accumulates [3, 15, 16]. We formalize this perspective as a unified skill lifecycle with five stages: **creation, memory, management, evaluation, and refinement**. This reframing turns skills from disposable artifacts into managed, testable, and transferable infrastructure: the foundation needed for agents to accumulate experience across tasks, sessions, and even across different agent systems.

MUSE-Autoskill framework. We instantiate this lifecycle in **MUSE-Autoskill Agent (Memory-Utilizing Skill Evolution; Figure 2)**. MUSE tightly couples skill creation with execution through a built-in `skill_create` tool invoked from within the runtime loop, eliminating the creation–usage mismatch. It introduces a multi-level memory comprising short-term, long-term, and (uniquely) skill-level memory, which accumulates per-skill experience across tasks and informs future invocations. An evaluation subsystem grounds reliability in unit tests and execution feedback, automatically triggering refinement when tests fail. A structured context manager with adaptive compression and cross-session state persistence supports long-horizon tasks without information loss or context-window blowup. Together, these components make skills externalized, testable, and transferable, rather than internal model behavior locked inside opaque weights.

Results. Figure 1 previews our headline results on **SkillsBench**, a benchmark of real-world tasks graded by automated verifiers in standardized Docker environments. On the 75-task common set, MUSE-Autoskill achieves the best no-skill accuracy (**46.95%**) and the best human-skill accuracy (**59.67%**, a +12.72 pp lift). Human skills improve all four GPT-5.5-backed agents by +10.78 to +13.73 pp. Self-created skills also improve every skill-creating agent on the same all-task denominator: MUSE-Autoskill reaches **53.42%**, above Codex (47.52%) and Claude Code (44.27%). A stability analysis in Appendix K further shows that MUSE-Autoskill-created skills have the lowest run-to-run reward dispersion among the compared self-created-skill conditions. Generated skills transfer across agent runtimes: Hermes reaches **51.90%** with MUSE-Autoskill-created skills, exceeding Hermes with human skills at 48.02%. We further evaluate on **SkillLearnBench**, a separate 20-task, 100-instance benchmark for continual skill generation, where MUSE-Autoskill also obtains the best human-skill accuracy (72.0%) and self-created-skill accuracy (48.0%).

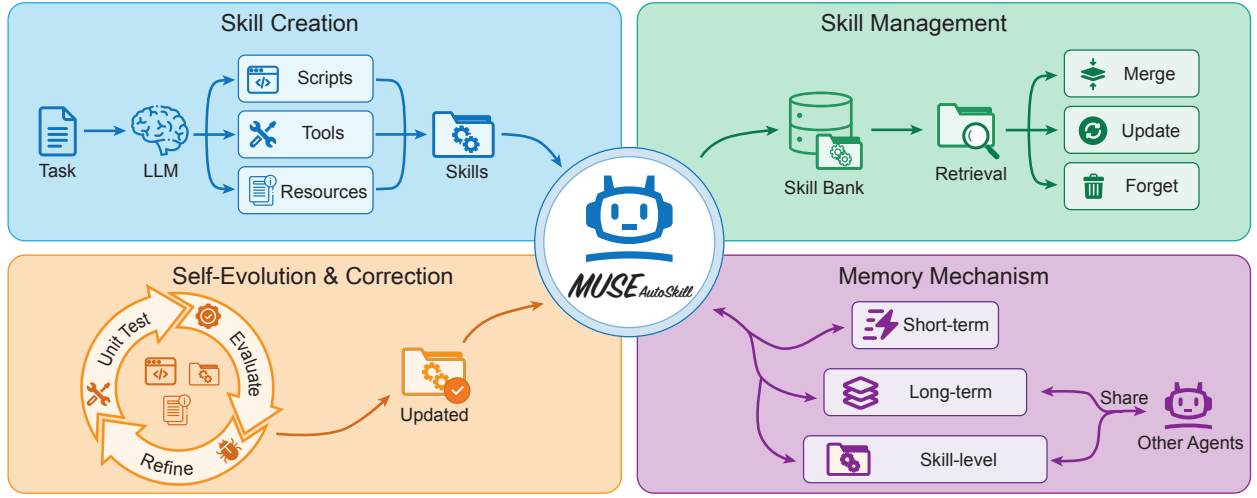


Figure 2 MUSE-Autoskill Agent architecture. MUSE organizes skills into a unified lifecycle of creation, memory, management, evaluation, and refinement, enabling agents to generate, refine, and reuse skills with accumulated experience over time.

Contributions. This paper makes four contributions:

- **Skill lifecycle.** We reframe skills from one-off generation outputs into long-lived, lifecycle-managed assets, identifying five stages (creation, memory, management, evaluation, refinement) that any practical skill-centric agent system must address.
- **MUSE-Autoskill.** A skill-centric agent that improves its task-solving capability over time by integrating skill creation with runtime execution, evaluating code-backed skills with unit tests and using sandbox/runtime feedback for all generated skills.
- **Infrastructure.** Multi-level memory with a novel skill-level memory that accumulates per-skill experience across tasks; adaptive context compression with cross-session state persistence; and cross-agent skill transfer that makes generated skills usable beyond their authoring agent.
- **Validation.** Best-in-class SkillsBench accuracy among four GPT-5.5-backed agents on the 75-task common set (59.67% with human skills, +12.72 pp lift); strongest self-created skill result under all-task scoring (53.42%); generated skills that transfer to Hermes above its human-skill score; and corroborating Skill-LearnBench results on 100 verified instances.

2 Related Work

2.1 LLM Agents

LLM-based agents that interact with tools, environments, and data have advanced rapidly in recent years [5, 6, 23, 31]. Building on ReAct [37]’s interleaving of reasoning and action, follow-up systems extend the paradigm to broader workflows, including multimodal autonomous agents such as Agent-Omni [11], MUSE [14], and OmniGAIA [10], and a wider body of work on self-improving agents [16, 27]. A parallel line of work focuses on equipping agents with tool-use capabilities, ranging from few-shot tool calling [22] to tool orchestration via model selection [24] and large-scale API retrieval [21]; for software engineering specifically, agents such as CodeAgent [38], SWE-Agent [34], and OpenHands [29] drive tool-integrated workflows over sandboxed shells and editors to resolve real-world repository tasks. The capabilities of such systems are now measured by general agent benchmarks including GAIA [17], SWE-bench [8], and AgentBench [13], which together cover web browsing, real-world software engineering, and multi-environment tool use. Despite this progress, most agent frameworks treat the set of available actions as either a fixed, hand-engineered tool registry or a flat

conversational scratchpad. They do not natively support agents that can author, validate, and accumulate their own reusable capabilities over time, which is precisely the gap the skill-centric literature, and our framework, set out to close.

2.2 Automatic Skill Systems

We organize the growing literature on automatic skill systems along two axes: which stages of the skill lifecycle (creation, memory, management, evaluation, refinement) a method addresses, and whether it operates entirely at inference time or requires additional model training. Table 1 summarizes the resulting comparison along these two axes.

The first major direction builds skill systems on top of pretrained LLMs without any fine-tuning. Voyager [28] is the seminal example: in the Minecraft setting, it maintains an ever-growing library of executable-code skills, with self-verification and iterative prompting that lets the same LLM both author and refine skills in response to environment feedback. Follow-up work generalizes this paradigm to general-purpose agents: AutoSkill [36] derives, maintains, and reuses skills from dialogue and interaction traces as a model-agnostic plugin layer; EvoSkill [1] analyses execution failures and proposes new skills or edits, retaining only those that improve held-out validation under a Pareto-frontier selection; and SkillGen [15] iteratively refines skills via contrastive induction over successful and failed trajectories, modelling each skill as an intervention to empirically verify its net effect. The feedback-driven refinement underlying these methods is rooted in a broader self-improvement literature outside the skill setting: Reflexion [27] maintains reflective text in an episodic memory buffer across attempts, Self-Refine [16] iteratively rewrites outputs using self-generated critiques, Self-Debug [3] closes the loop on code generation with execution and unit-test traces, and ExpeL [39] extracts natural-language insights across training tasks for inference-time reuse. These methods all improve agent behavior through linguistic feedback but stop short of treating skills as first-class, externalized, testable artifacts that outlive a single task or agent. On the industrial side, Anthropic’s Agent Skills [2] standardize skills as portable folders of SKILL.md instructions and scripts loaded via progressive disclosure; this is the closest practical analogue of our externalized skill format, but the system leaves evaluation and refinement to human authoring. Collectively, these training-free methods are lightweight and naturally portable across LLM backbones, yet each covers only part of the lifecycle: none simultaneously supports structured per-skill memory, unit-test-driven evaluation, and automatic refinement triggered by test feedback.

A second, concurrent direction uses reinforcement learning to optimize skill behavior jointly with the policy. SkillMaster [35] learns a single policy that both acts and edits its skill bank, with edits credited by counterfactual downstream utility. Skill1 [25] frames skill evolution as a unified RL problem, co-optimizing skill selection, utilization, and distillation under a shared task-outcome reward. SkillOS [18] pairs a frozen executor with a trainable curator that updates an external skill repository from accumulated experience, and shows that the curator generalizes across executor backbones; this is a portability axis complementary to ours, where the skills themselves rather than the curator are the unit of transfer. Youtu-Agent [26] pursues a related direction via hybrid policy optimization of tools and agent configurations. These RL-based methods can attain strong optimality on the environments they are trained on, but they couple skill behavior to a trained policy or curator: migrating to a new backbone typically requires additional training, and skills produced by one trained policy are not directly usable by a different agent without re-training.

2.3 Benchmarks and Positioning

Several recent benchmarks complement the methods above by stressing different lifecycle stages. Skills-Bench [9], which we adopt in our experiments, measures end-to-end task accuracy with and without skills across diverse Docker-evaluated real-world tasks. SkillRet [4] isolates the management stage by evaluating skill retrieval at scale from a library of nearly 18,000 community-contributed skills. SkillLearnBench [41] and LifelongAgentBench [40] focus on continual and lifelong skill acquisition over task streams, and notably report that strong individual methods do not consistently dominate, motivating system-level designs such as ours. A concurrent survey [33] catalogues skill-acquisition modalities and architectural choices for LLM agents, situating both training-free and training-based directions within a broader taxonomy.

Compared with the methods above, MUSE-Autoskill differs in that it brings all five lifecycle stages together

Table 1 Related work on automatic skill systems by lifecycle stage. ✓ = covered; △ = partial; ✗ = not addressed. Memory = persistent per-skill experience across tasks. Cross-agent = skills from one agent are usable by another without modification; ✓ requires an explicit cross-agent transfer experiment, △ indicates portability only across LLM backbones or product variants of the same agent. Training-free = inference-time only, no fine-tuning or RL.

Method	Lifecycle stage					Cross-agent	Training-free
	Creation	Memory	Management	Evaluation	Refinement		
Voyager [28]	✓	✗	✓	△	✓	△	✓
AutoSkill [36]	✓	△	△	✗	✓	✗	✓
EvoSkill [1]	✓	✗	△	✓	✓	✗	✓
SkillGen [15]	✓	✗	✗	✓	✓	△	✓
Anthropic Skills [2]	✓	✗	✓	✗	✗	△	✓
SkillMaster [35]	✓	✗	△	△	✓	✗	✗
Youtu-Agent [26]	✓	✗	△	✗	✗	✗	✗
Skill1 [25]	✓	△	△	△	△	✗	✗
SkillOS [18]	✓	✓	✓	△	✓	✗	✗
MUSE-Autoskill (Ours)	✓	✓	✓	✓	✓	✓	✓

within a single training-free framework, rather than addressing creation or refinement in isolation. In particular, it introduces skill-level memory that accumulates per-skill experience across tasks, uses unit-test-driven evaluation that automatically triggers refinement when tests fail, and is the only general-purpose method to empirically validate cross-agent skill transfer by injecting its generated skills into a different agent without modification (Section 4); other portability claims in the literature are limited to swapping the underlying LLM backbone or sharing skills across product variants of the same agent family, without an explicit cross-agent experiment. The combination of full lifecycle coverage and a training-free design also makes the system portable across LLMs and agent architectures, as summarized in the bottom row of Table 1.

3 MUSE-Autoskill Agent

In this section, we present **MUSE-Autoskill Agent**, a skill-centric agent framework that solves complex tasks by dynamically creating, reusing, and refining skills. MUSE integrates skill creation, execution, memory, management, and evaluation within a unified agent loop. Figure 2 illustrates the overall architecture and the five lifecycle stages described below.

3.1 Agent Framework

The agent operates in an iterative decision-making loop consisting of three core stages: *Planning*, *Action*, and *Observation* [37]. Given an input query, the agent continuously cycles through these stages to progressively solve the task. This design enables dynamic reasoning, skill invocation, and adaptive refinement based on intermediate feedback from tool calls and skill executions.

Planning In the planning stage, the agent interprets the input query and determines the next step toward achieving the task objective. This involves decomposing the problem, selecting appropriate strategies, and deciding whether to invoke external skills. The agent may also leverage past observations and memory to refine its plan, enabling more informed and context-aware decisions.

Action In the action stage, the agent executes the planned step by invoking skills. These may include retrieving existing skills from the skill bank or utilizing built-in functions such as skill creation and web search. The selected skill is invoked within the agent’s ReAct loop using its built-in tools, producing intermediate or final outputs for the task. The detailed execution mechanism of skills will be introduced in Section 3.2.

Observation In the observation stage, the agent collects and analyzes the results returned from execution. These observations are used to evaluate progress toward the goal and to inform subsequent planning decisions. Through this feedback loop, the agent can iteratively refine its behavior, handle errors, and adapt to complex, multi-step tasks in practice.

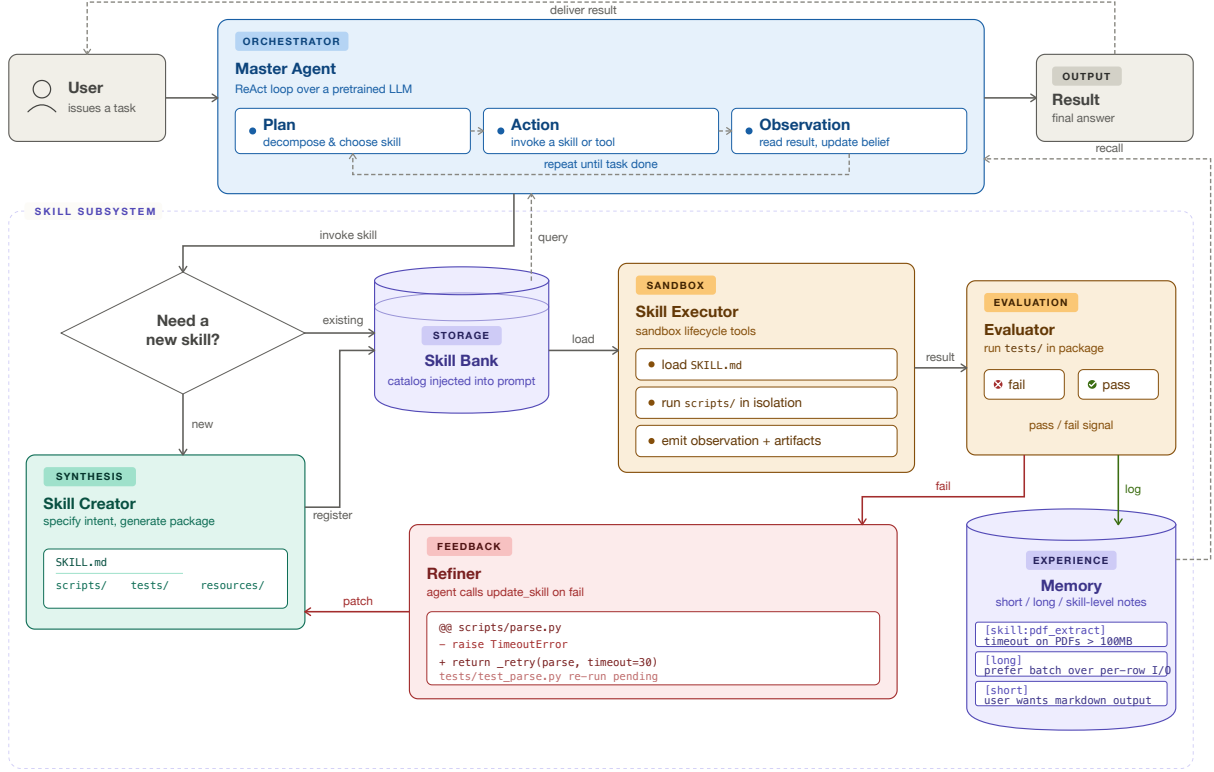


Figure 3 End-to-end flow of MUSE-Autoskill. The Master Agent runs a ReAct loop; when a skill is needed it either retrieves one from the Skill Bank or dispatches the Skill Creator to synthesize a new package (SKILL.md plus optional `scripts/` and, for code-backed skills, `tests/`). The Evaluator runs bundled tests when present and otherwise relies on sandbox/runtime checks; failed checks send the package to the Refiner, while accepted observations are appended to Memory and surfaced on later steps.

3.2 Skill Lifecycle

As illustrated in Figure 3, the agent organizes skills into a unified lifecycle of five stages: *creation*, *memory*, *management*, *evaluation*, and *refinement*. To bootstrap this process, the agent is equipped with a small set of built-in skills, including `skill_create` and `web_search`. In the autonomous self-creation setting, new task-specific skills are produced through this mechanism; in evaluation settings, the same runtime can also load externally supplied human skills through the skill bank.

Skill As illustrated in Figure 2, a *skill* is the basic unit of execution in our system. Each skill is packaged as a structured directory with standard components, following Anthropic’s Agent Skills format [2]. It includes a SKILL.md file that defines its interface, such as name, description, inputs, and outputs, and may also include subdirectories like `scripts/` for executable code, `resources/` for auxiliary data, and, when the skill includes generated code, `tests/` for validation.

Skills are executed through a unified interface. At runtime, the agent reads SKILL.md to understand how to use the skill, and decides whether to read resources, run scripts, or both. If scripts are required, the execution engine runs the corresponding code with the given inputs and returns the outputs.

Using skills changes repeated work from open-ended reasoning into a shorter procedure call. The agent can load only the skill interface first, then read the full body or run bundled scripts only when needed; reused skills therefore reduce repeated exploration and make later runs more direct.

Skill Creation As illustrated in Figure 2, new skills are generated through the built-in `skill_create` skill. When existing skills are not sufficient, the agent provides a high-level specification of the desired functionality,

including its purpose, inputs, and expected outputs.

Based on this specification, the system follows a structured pipeline to construct the skill. It first generates the `SKILL.md` file to define the interface, then plans the internal structure such as `scripts/`, `resources/`, and, for code-backed skills, `tests/`, and finally generates the corresponding files. The result is a complete and executable skill package.

After creation, the skill is checked before registration. Code-backed skills may include a `tests/` directory; when present, the system runs those tests inside the sandbox and blocks registration until they pass. For text-only procedural skills and code-backed packages without generated tests, the system falls back to sandbox execution checks and runtime feedback from the source trajectory. If a check fails, the agent inspects the error trace and invokes `update_skill` to patch the package before rechecking it. This create → evaluate → register loop keeps generated skills inspectable and gives the agent a concrete failure signal for later refinement.

Skill Evaluation As illustrated in Figure 2, skills are evaluated before they are reused. For code-backed skills, the strongest signal comes from unit tests when the package contains a `tests/` directory. For text-only procedural skills, and for code-backed skills without generated tests, the evaluator uses sandbox execution, source-trajectory checks, and runtime feedback as weaker but still useful validation signals.

This process filters out many incorrect or unstable packages and records the reason for failure. As part of the self-evolution loop shown in Figure 2, failed tests or failed execution checks can trigger updates or regeneration of the skill. The result is not a guarantee of correctness, but a concrete audit path: each accepted skill has either passed generated tests or survived the available sandbox/runtime checks.

Skill Execution As illustrated in Figure 2, skill execution is carried out within the agent’s ReAct loop using its built-in tools. Given a task, the agent reads the available skill catalog and selects an appropriate skill. It then reads the `SKILL.md` file to understand the skill interface, standard operating procedure, and required components.

Following the procedure defined in `SKILL.md`, the agent decides whether to read from `resources/`, execute code in `scripts/` via sandbox tools, or combine both. Code execution is mediated by a small set of sandbox lifecycle tools (`create_sandbox`, `sandbox_run`, `sandbox_upload/sandbox_download`, and `close_sandbox`) that the agent invokes from inside its ReAct loop. Each sandbox is an isolated process / container with its own filesystem, so failures, side effects, and resource usage are contained per skill invocation. Rather than introducing a separate execution engine, skill execution reuses the same general-purpose tools the agent already uses (file reading, terminal commands, sandbox calls), which avoids redundant infrastructure and lets execution benefit from the agent’s full reasoning capability.

The execution process is iterative: intermediate results are fed back into the agent’s reasoning loop, enabling progressive refinement and error handling. This unified approach ensures consistent execution across all skills while preserving flexibility for both simple and complex tasks.

Skill Memory As illustrated in Figure 2, the agent maintains memory at multiple levels to support skill reuse and accumulation over time. In particular, skill-level memory stores the skills themselves along with their metadata, such as descriptions, inputs, and usage history. This allows the agent to efficiently retrieve relevant skills for new tasks.

In addition, the agent appends notes and observations to short-term and long-term memory, providing context for future decisions. These notes help the agent avoid recreating a skill it has already used, remember file-format or environment quirks, and choose a known procedure before starting from scratch.

Skill Management As illustrated in Figure 2, skill management maintains the quality and usability of the skill bank. Each skill is indexed using metadata from `SKILL.md`, including its name, description, inputs, and outputs. At the start of each task, the agent is provided with a catalog of available skills injected into the system prompt, following the progressive-disclosure pattern of Anthropic’s Agent Skills [2]. The agent then

selects the most relevant skill during planning based on this catalog, enabling efficient reuse and reducing unnecessary skill creation.

In addition to retrieval, the system supports maintenance of the skill bank through three mechanisms: *refinement*, *merging*, and *pruning*. When a skill fails generated tests or produces incorrect outputs during execution, the agent revises or regenerates it based on the error feedback. When newly created skills overlap significantly with existing ones, the agent can merge them into a single, more general skill to avoid redundancy. Skills that consistently fail or remain unused over time can be removed from the active catalog. These operations keep the catalog smaller and make skill selection less dependent on scanning redundant entries.

3.3 Memory

Memory plays a central role in enabling MUSE to accumulate knowledge and reuse previously acquired capabilities. Our design builds on prior hierarchical memory architectures for LLM agents: MemGPT [19] pages between in-context and external memory in an OS-style hierarchy, Generative Agents [20] maintain a memory stream with periodic synthesis into higher-level reflections, and Reflexion [27] and ExpeL [39] accumulate natural-language reflections and insights across episodes. MUSE extends these by adding a per-skill memory scope tied to each `SKILL.md` file, complementing short- and long-term layers shared with prior work.

Skill-level Memory Each skill in the bank carries its own `.memory.md` file, into which the agent appends notes, lessons, and usage observations accumulated across tasks (e.g., known failure modes, input format quirks, performance caveats). When the same skill is loaded later, this per-skill memory is surfaced alongside its `SKILL.md` interface, letting the agent benefit from previously learned experience without re-deriving it.

Short-term Memory Short-term memory maintains the current task context, including intermediate reasoning steps, observations, and temporary execution results. As the context grows, it is adaptively compressed by summarizing intermediate steps, allowing the agent to handle long-horizon tasks without exceeding the model’s token budget during extended runs.

Long-term Memory Long-term memory stores persistent notes the agent appends across sessions, including reusable conclusions, environment quirks, and general lessons learned outside any single skill (e.g., “prefer batched I/O,” “the project uses pinned package versions”). Unlike short-term memory, long-term memory is not subject to compression and serves as a growing repository of accumulated experience, enabling the agent to improve decision-making over time by drawing on lessons learned in prior runs.

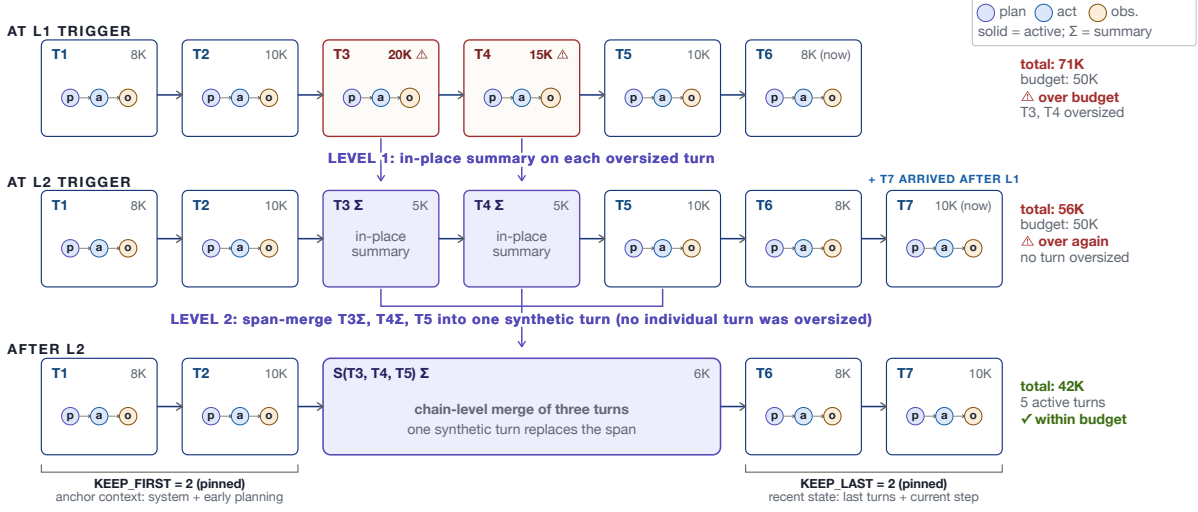
3.4 Context Management

The agent maintains context as a DAG of conversation nodes, one per turn (Figure 4). Each node records the model response, tool calls, observations, and per-call token usage from one step. Every node carries two sets of pointers: a mutable `parent_id` that defines the current active chain sent to the LLM, and an immutable `history_prev/history_next` pair that defines the full history of original turns. The active chain is always a sub-graph of the full history.

As tasks grow longer, the accumulated short-term context can exceed the model’s token budget. Existing remedies span token-level prompt compression [7], attention-sink-based KV retention for streaming inference [32], and OS-style virtual context management for general LLM agents [19]; positional studies further document significant degradation when relevant content is buried in the middle of a long context [12], which motivates the explicit first/last pinning we adopt below. To handle this at the agent level, MUSE applies adaptive context compression with two levels. **Level-1** (single-node compression) scans the active chain for individual nodes whose token footprint exceeds a per-node threshold (typically a large tool output or a verbose observation) and replaces that node’s content with a compact summary while keeping it in the chain. If the total context is still over budget after Level-1, **Level-2** (chain compression) merges a contiguous range of intermediate nodes into a single synthetic summary node, which then takes the place of those nodes in the active chain. We always try Level-1 first because it is the strictly less destructive operation: only the

Context management: DAG of ReAct turns with two-level adaptive compression

each turn is a (plan, action, observation) triple chained left-to-right; L1 rewrites individually oversized turns; L2 merges the compressible span when L1 is not enough



All compression operates on the active chain only; the original 7 turns remain in the full history (linked by immutable `history_prev` / `history_next` pointers), so any prior state can be replayed or resumed across sessions.

Figure 4 Adaptive context compression over a DAG of ReAct turns. Each turn is a (plan, action, observation) triple; the first `KEEP_FIRST` and last `KEEP_LAST` turns are always pinned and only the middle is eligible for compression. **Top→Middle:** Level-1 rewrites individually oversized turns in place. **Middle→Bottom:** when no single turn is oversized but the chain is still over budget, Level-2 merges the compressible span into one synthetic node. Original turns remain in the full history (linked by immutable `history_prev`/`history_next` pointers), so the trajectory is fully replayable.

offending node’s payload is rewritten, while the per-turn boundaries and the full plan/action/observation structure of the chain are preserved, so downstream turns can still reference earlier turns by their original positions. Level-2 collapses several turns into one synthetic node and loses that per-turn structure, so we invoke it only when single-node summaries alone cannot bring the chain under budget. In both levels the original nodes remain in the full history, so the active chain is always recoverable. Long-term memory and the skill bank, by contrast, are stored separately and are not subject to compression, allowing the agent to accumulate experience across sessions without loss.

In addition, the agent’s full state, including conversation history, skill usage records, and execution metadata, is persisted as a snapshot after each session. This allows tasks to be resumed from an intermediate state without restarting from scratch, which is essential for complex, long-horizon workflows that may span multiple sessions.

4 Experiments

We first conduct experiments on SkillsBench to evaluate three aspects of our framework: whether skill usage improves agent performance, whether MUSE-Autoskill can automatically generate effective skills from its own experience, and whether generated skills can transfer across agents. We then evaluate the same agent family on SkillLearnBench to test whether the skill-use and skill-creation trends hold on an independent continual-skill benchmark.

4.1 Experimental Setup

Benchmarks We evaluate on two benchmarks designed for skill use and skill creation. **SkillsBench** [9] assesses AI agents on real-world tasks that require domain-specific knowledge and tool use. Each task runs

Table 2 Main accuracy results (%) on SkillsBench and SkillLearnBench. SkillsBench uses the strict 75-task $\times$ 5-run denominator; SkillLearnBench uses 100 verified instances. Self-created skills are evaluated only for self-creating agents. **Bold** = best within each benchmark column; **blue rows** = MUSE-Autoskill (ours).

Benchmark	Agent	Without Skills	Human Skills	Self-Created Skills
SkillsBench	Hermes	37.24%	48.02%	—
SkillsBench	Codex	44.80%	57.58%	47.52%
SkillsBench	Claude Code	42.43%	56.15%	44.27%
SkillsBench	MUSE-Autoskill (Ours)	46.95%	59.67%	53.42%
SkillLearnBench	Hermes	37.0%	70.0%	—
SkillLearnBench	Codex	39.0%	68.0%	40.0%
SkillLearnBench	Claude Code	33.0%	63.0%	37.0%
SkillLearnBench	MUSE-Autoskill (Ours)	43.0%	72.0%	48.0%

in an isolated Docker container and is graded by an automated verifier that checks only the final output files, assigning a reward in $[0, 1]$. We use the 75-task common set listed in Appendix A, grouped into four super-domains: science & engineering, data analysis, document processing, and ops & planning.

We further evaluate on **SkillLearnBench** [41], an independent benchmark designed for continual skill generation on real-world agent tasks. It contains 20 skill-dependent tasks across software engineering, information retrieval, productivity tools, data and analytics, content and creative work, and utility workflows. Each task has multiple verified instances, yielding 100 instances in total; each instance is also executed in an isolated Docker environment and graded by an automated verifier over the submitted artifacts.

Agents and Models We evaluate four agents, all using GPT-5.5 as the backbone model: **MUSE-Autoskill** (our method), **Codex**, **Claude Code**, and **Hermes**. Since all agents share the same underlying model, performance differences reflect agent system design (including tool strategies, context management, planning, and skill usage) rather than model capacity.

Model Backend In all experiments, Hermes, Codex, Claude Code, and MUSE-Autoskill use the same GPT-5.5 deployment, `gpt-5.5-2026-04-24` (reported as GPT-5.5 (04/24/2026)); in particular, Claude Code’s model calls are routed to this GPT-5.5 deployment through a compatibility bridge. We did not override decoding parameters such as temperature or top- p , so provider defaults are used throughout. Thus, performance differences reflect the surrounding agent systems, including prompts, tool loops, context handling, and skill-loading mechanisms, rather than different model backbones.

Evaluation Protocol For SkillsBench, each agent-task-configuration combination is run 5 times independently in separate Docker containers. We report all-task accuracy over 375 runs: the sum of rewards divided by 75×5 . For SkillLearnBench, each of the 100 verified instances is evaluated once, and accuracy is the number of successful instances divided by 100. In both benchmarks, self-created-skill settings use the same denominator as their corresponding no-skill and human-skill settings; tasks or instances without a generated skill contribute 0 rather than being backfilled with the no-skill result.

4.2 Effect of Skill Usage

Setup We compare each agent under two shared conditions: *without skills* (the agent relies solely on its own knowledge) and *with human skills* (benchmark-provided, human-authored skills are injected into the agent’s workspace at task start). For Codex, Claude Code, and MUSE-Autoskill, Table 2 also reports the headline self-created-skill result under the same benchmark denominator. Hermes is included only for the without-skill and human-skill comparison in SkillLearnBench.

Results Table 2 summarizes the main results on both benchmarks. Human skills improve every agent on both benchmarks. MUSE-Autoskill achieves the highest no-skill, human-skill, and self-created-skill accuracy in both settings, reaching 59.67% with human skills on SkillsBench and 72.0% on SkillLearnBench.

Table 3 Per-domain accuracy (%) under *without skills* (w/o) and *with human skills* (w/ hum) conditions on the 75-task common set. **Bold** = best in each row’s human-skill columns; **blue columns** = MUSE-Autoskill (ours).

Domain	#	Hermes		Codex		Claude Code		MUSE-Autoskill (Ours)		Best
		w/o	w/ hum	w/o	w/ hum	w/o	w/ hum	w/o	w/ hum	
Science & Engineering	18	40.61	58.89	54.52	67.12	52.33	63.88	54.57	67.97	Claude Code
Data Analysis	18	36.34	40.60	38.31	50.18	38.05	52.59	39.49	51.48	
Document Processing	14	54.29	58.57	68.57	72.86	62.86	64.29	67.14	74.29	Ours
Ops & Planning	25	25.92	39.64	29.16	47.48	27.00	48.60	35.52	51.40	Ours
Overall	75	37.24	48.02	44.80	57.58	42.43	56.15	46.95	59.67	Ours

Table 4 Self-created skill performance on the 75-task common set. Uncovered tasks are counted as 0 in all-task accuracy. Covered-task accuracy is reported only to diagnose skill quality conditional on successful generation. **Bold** = best in column; **blue row** = MUSE-Autoskill (ours).

Agent	Covered	Uncovered	All-75 Acc.	Covered Acc.	Lift vs. w/o
Codex	47	28	47.52%	75.83%	+2.72 pp
Claude Code	44	31	44.27%	75.45%	+1.84 pp
MUSE-Autoskill (Ours)	47	28	53.42%	85.24%	+6.47 pp

Discussion The consistent improvement across agents suggests that the skill mechanism itself is effective under strict denominators that count operational failures as zero. MUSE-Autoskill leads both no-skill and human-skill settings, while Codex and Claude Code are close when human skills are available. SkillLearn-Bench shows a larger human-skill lift than SkillsBench because it is explicitly built around skill-dependent instances, but it also shows the same automatic skill-generation bottleneck: self-created skills improve over the no-skill condition, yet remain below human-authored skills.

Per-Domain Breakdown SkillsBench’s per-task category metadata is free-text, so we group the 75 common-set tasks into four super-domains: **Science & Engineering** (18 tasks), **Data Analysis** (18), **Document Processing** (14), and **Ops & Planning** (25). Appendix Table 9 lists the task-level category and assigned super-domain for every task. Figure 1 and Table 3 report accuracy in each domain under both conditions. MUSE-Autoskill achieves the highest human-skill score in 3 of 4 domains and overall; Claude Code is strongest in Data Analysis.

4.3 Automatic Skill Generation

Setup We analyze automatic skill generation in detail on the 75-task SkillsBench common set. The process follows a two-phase protocol. In **Phase 1**, the agent solves each task without skills. For tasks where a usable source trajectory is available, we invoke the agent’s skill-creation mechanism to distill it into a `SKILL.md` and optional helper scripts. In **Phase 2**, the generated skill is injected back and the same agent is re-evaluated for 5 runs. Tasks without a usable generated skill are not dropped; they are counted as 0 in the 75-task denominator.

Results Table 4 compares self-created skill performance for Codex, Claude Code, and MUSE-Autoskill. Self-created skills improve all three agents under the strict all-75 metric. MUSE-Autoskill obtains the largest gain and the strongest self-created result, improving from 46.95% to **53.42%**.

Discussion On covered tasks, generated skills are highly effective: MUSE-Autoskill reaches 85.24%, above its 81.17% human-skill accuracy on the same subset, while Codex and Claude Code reach 75.83% and 75.45%. This suggests that once a successful trajectory can be converted into a reusable skill, the resulting lifecycle-managed skill can match or exceed benchmark-provided human guidance. The all-task scores are lower because 28–31 tasks still have no usable generated skill and therefore contribute 0. The primary bottleneck is therefore coverage: generating usable skills for more tasks, not only improving the skills that already exist.

Table 5 Cross-agent transfer into Hermes on the 75-task common set. Covered tasks use transferred-skill runs; uncovered, missing, and error runs are counted as 0. **Bold** = best non-baseline transfer result.

Hermes Configuration	Covered	Uncovered	All-75 Acc.	Delta vs. w/o
Without skills	75	0	37.24%	–
Human skills	75	0	48.02%	+10.78 pp
With Codex-created skills	47	28	37.01%	-0.23 pp
With Claude Code-created skills	44	31	45.97%	+8.73 pp
With MUSE-Autoskill-created skills	47	28	51.90%	+14.66 pp

4.4 Cross-Agent Skill Transfer

Setup We test whether generated skills can benefit a different agent. We inject skills created by Codex, Claude Code, and MUSE-Autoskill into Hermes without task-specific modification, and evaluate Hermes on the 75-task common set. The source generated-skill banks cover 47 Codex-created-skill tasks, 44 Claude Code-created-skill tasks, and 47 MUSE-Autoskill-created-skill tasks. Tasks outside the corresponding source-skill coverage are counted as 0, matching the strict all-task scoring protocol above.

Results Table 5 summarizes the results. Here, “Covered” is the number of source generated-skill tasks used for transfer, while “Uncovered” is the number of remaining tasks counted as 0. MUSE-Autoskill-created skills transfer best to Hermes: they raise Hermes from 37.24% without skills to **51.90%**, which is also above Hermes with human skills at 48.02%.

Discussion MUSE-Autoskill-created skills transfer most effectively under the all-75 metric, improving Hermes by +14.66 pp and exceeding the human-skill condition by 3.88 pp. Claude Code-created skills also improve Hermes to 45.97%, while Codex-created skills reach 37.01% and do not improve over the no-skill baseline in this run. Transfer is therefore a joint function of skill representation quality and coverage; MUSE-Autoskill and Codex have the same 47-task source coverage here, but MUSE-Autoskill-created skills are substantially more effective when transferred.

Skill Generation and Usage Cost Table 6 compares the one-time cost of generating a skill and the per-task cost of reusing it for Codex, Claude Code, and MUSE-Autoskill. Rows are computed on each agent’s own covered subset (47 Codex tasks, 44 Claude Code tasks, and 47 MUSE-Autoskill tasks), so the table diagnoses reuse economics rather than replacing the all-75 accuracy metric in Table 4. MUSE-Autoskill has the lowest one-time token cost (364K) and is the only agent whose self-created skills exceed its human-skill covered-subset accuracy while reducing both median tokens and latency. Claude Code self-created skills are cheaper but slightly less accurate than human skills, while Codex self-created skills are faster but substantially more token-heavy. The final block is a transfer diagnostic: Hermes remains the Hermes agent, but uses the MUSE-Autoskill-created skill bank on the MUSE-covered subset; token totals are omitted for these Hermes transfer rows because the current logs do not expose normalized token totals.

4.5 Generated Skill Capabilities

Aggregate Performance Figure 5 plots mean reward against cost for Codex, Claude Code, and MUSE-Autoskill on each agent’s own self-created-skill covered subset. Each arrow starts from the no-skill condition: dashed arrows point to the human-skill condition, while solid arrows point to the self-created-skill condition. MUSE-Autoskill shows the strongest pattern: self-created skills improve covered-task reward from 74.92% without skills and 81.17% with human skills to 85.24%, while reducing median latency to 434.7 s and median tokens to 499K. Claude Code self-created skills are cheaper than its human-skill condition, but slightly lower in reward (75.45% vs. 77.75%). Codex self-created skills improve over its no-skill baseline and cut latency sharply, but they consume more tokens than both no-skill and human-skill runs. The all-75 score remains lower because generated-skill coverage is incomplete; this is the coverage bottleneck discussed below.

Table 6 Self-created skill generation and usage cost. Self-created-agent rows use each source agent’s own covered subset; Hermes rows use the 47-task MUSE-covered subset. Tokens are median per-run totals when parseable; Hermes transfer-row token totals are omitted because the current logs do not expose normalized token totals. **Blue rows** = MUSE-Autoskill-created skills (ours).

Agent	Configuration	Covered	Acc.	Tokens	Latency (s)	Turns
Self-created skill lifecycle cost (source agent’s covered subset)						
Codex	One-time skill creation	47	–	446K	229.4	20
Codex	Without skills	47	71.5%	325K	944.5	12
Codex	Human skills	47	76.6%	365K	794.7	14
Codex	Self-created skills	47	75.8%	922K	423.1	25
Claude Code	One-time skill creation	44	–	557K	147.2	16
Claude Code	Without skills	44	71.3%	503K	294.0	18
Claude Code	Human skills	44	77.8%	625K	248.7	21
Claude Code	Self-created skills	44	75.5%	352K	159.0	14
MUSE-Autoskill	One-time skill creation	47	–	364K	156.3	6
MUSE-Autoskill	Without skills	47	74.9%	579K	729.3	20
MUSE-Autoskill	Human skills	47	81.2%	638K	755.7	20
MUSE-Autoskill	Self-created skills (ours)	47	85.2%	499K	434.7	15
Transfer usage on the MUSE-covered subset						
Hermes	Without skills	47	59.0%	–	336.0	14
Hermes	Human skills	47	64.3%	–	342.9	13
Hermes	With MUSE-created skills (ours)	47	82.8%	–	262.9	13

Case Studies We highlight three skills where the generated artifact carries non-trivial domain knowledge, plus one regression.

(i) *adaptive-cruise-control* requires a discrete PID controller satisfying verifier constraints on overshoot, steady-state error, and rise time. MUSE-Autoskill without skills achieves 40% (2 of 5 runs). The generated skill `implement-acc-simulation` codifies the discrete PID equation, anti-windup, gain-tuning heuristics, and the JSON file format required by the verifier; self-created accuracy reaches **100%**. Hermes using the same MUSE-created skill improves from 20% to 60%, confirming that the skill transfers domain knowledge rather than memorizing the MUSE runtime.

(ii) *flink-query* asks the agent to author an Apache Flink Java job that reads gzipped Google ClusterData traces, performs microsecond event-time sessionization, and emits tuples in an exact format. The baseline solves only one of five runs (20%) because the agent cannot recover the project’s POJO and `AppBase` skeleton conventions from documentation alone within its turn budget. The generated skill `implement-clusterdata-flink-session-query` packages the schema parsing, the `clusterdata.utils.AppBase` extension protocol, event-time session triggers, and a Maven-based validation recipe with synthetic gzipped data; Phase 2 reaches **100%** across all five runs.

(iii) *weighted-gdp-calc* requires filling an Excel workbook with two-condition lookups and `SUMPRODUCT`-based weighted means while preserving existing formatting and avoiding macros/VBA. The generated skill `excel-financial-formula-modeling` names `openpyxl` as the right tool, lists the formula patterns, and adds a verification step that recomputes target cells from source data; the baseline jumps from 20% to **100%**. Notably, the same skill description guides Hermes through the identical workflow without modification.

(iv) *Regression: hvac-control*. The largest MUSE self-created regression (80% $\rightarrow$ 20%) occurs on a task that requires PI control of a first-order thermal simulator. The source trajectory used a calibration window and gain-estimation routine specific to that simulator’s noise profile; when re-applied in fresh runs, the variance in calibration data occasionally produces tuned gains outside the verifier’s stability margin. This is a case where the skill encodes a procedure that worked once but is less robust than baseline trial-and-error, and motivates the audit finding (next subsection) that some skills carry source-trajectory-specific assumptions that limit out-of-distribution robustness.

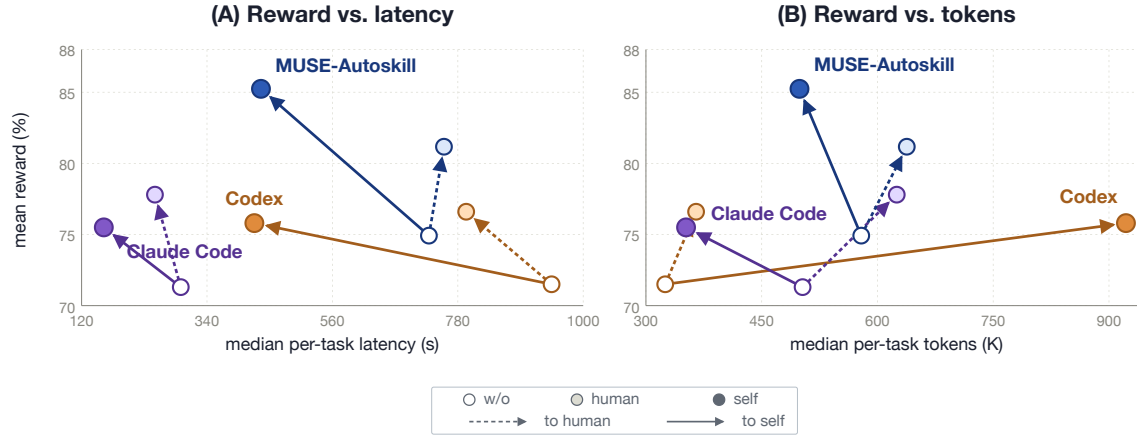


Figure 5 Generated-skill tradeoffs on covered tasks. Mean reward and median cost for Codex, Claude Code, and MUSE-Autoskill on each agent’s own self-created-skill covered subset. **(A)** Reward vs. median per-task latency. **(B)** Reward vs. median per-task tokens. Open circle = without skills; light fill = human skills; solid fill = self-created skills. Dashed arrows point from no skills to human skills; solid arrows point from no skills to self-created skills.

Table 7 Skill package anatomy on the 75-task common set. Line count and size are medians over `SKILL.md`; directory shares are package percentages and are not mutually exclusive.

Source	Packages	Tasks	Lines median	IQR	Size KB	Only SKILL.md	scripts/	tests/	refs/
Human-authored SkillsBench	189	75	165	89–289	5.2	65.1%	27.0%	0.0%	13.2%
Codex-created	50	47	128	98–195	7.0	14.0%	46.0%	0.0%	16.0%
Claude Code-created	47	44	89	72–109	5.6	19.1%	72.3%	0.0%	12.8%
MUSE-Autoskill-created	50	47	310	240–412	13.2	90.0%	8.0%	8.0%	0.0%

4.6 Analysis

Skill Quality Audit We manually inspect the 50 MUSE-generated skill packages covering 47 tasks for potential benchmark leakage. None of the inspected skills hardcode expected verifier outputs, branch on task identifiers, or read from ground-truth files. A subset of skills contain benchmark-specific assumptions such as fixed file names, directory paths, or numerical ranges derived from the source run. These do not constitute cheating but may limit generalization to out-of-distribution inputs. The structural distribution of agent-created and human-authored skill packages is summarized in Table 7.

Skill Anatomy and Distribution Figure 6 and Table 7 compare agent-created skill packages from Codex, Claude Code, and MUSE-Autoskill against 189 human-authored SkillsBench packages from the same 75-task common set. MUSE-Autoskill-created skills are longer than the other generated baselines and human-authored skills, but the extra length is concentrated in procedural detail: input/output schemas, verifier-facing file conventions, failure modes, and validation steps. Codex and Claude Code more frequently bundle helper scripts, whereas MUSE-Autoskill more often distills a self-contained textual procedure and uniquely generates test packages in this audit.

Reward–Cost Relationship Human skills raise reward for every agent while reducing median wall-clock latency (Figure 7A). This is the main cost-quality pattern: skills add context, but they also replace exploratory reasoning with a more direct procedure. Turn counts do not uniformly decrease, so the robust claim is latency improvement rather than fewer ReAct steps. Hermes, Codex, Claude Code, and MUSE-Autoskill all move to higher reward and lower median latency under human skills; full latency percentiles for all agents are reported in Appendix J.

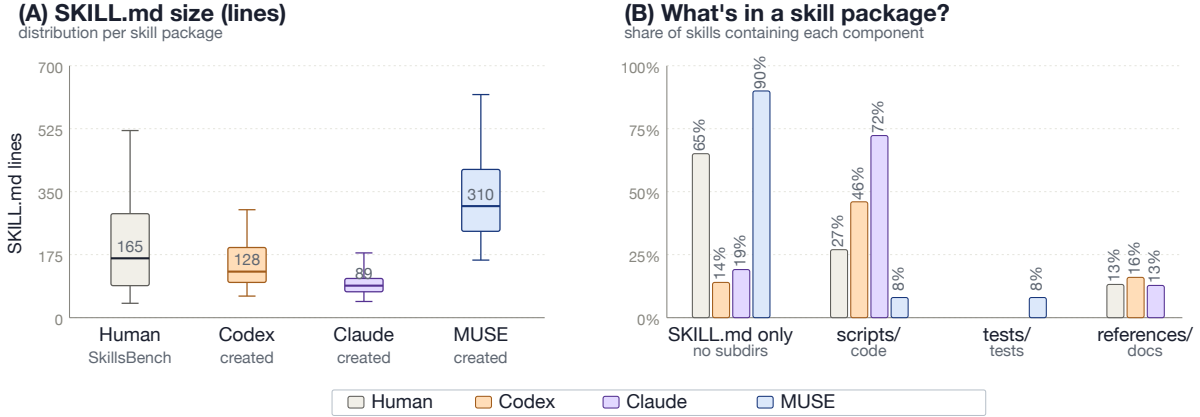


Figure 6 Skill anatomy: human-authored and agent-created packages. (A) SKILL.md line counts on the 75-task common set: MUSE-Autoskill-created skills are longest (median 310 lines), compared with human-authored SkillsBench skills (165), Codex-created skills (128), and Claude Code-created skills (89). (B) Share of skill packages containing each component. Codex and Claude Code more often emit helper `scripts/`; MUSE-Autoskill more often emits self-contained SKILL.md-only packages and is the only group with generated `tests/` packages in this audit.

Table 8 Token usage diagnostic on the 75-task common set. Totals are median per-run tokens where parseable. Fresh/cache decomposition is reported separately in Appendix H.

Agent	Total w/o	Total human	Δ total
Hermes	483K	406K	-16.0%
Codex	342K	361K	+5.4%
Claude Code	590K	651K	+10.3%
MUSE-Autoskill	579K	638K	+10.1%

Token Usage Token accounting is diagnostic because normalized fresh/cache traces are not available for every runtime. For the agents with parseable totals, skills increase reward while changing the median token footprint in different directions (Figure 7B and Table 8). Codex gains +12.78 pp for a +5.4% median-token increase. MUSE-Autoskill gains +12.72 pp for a +10.1% increase, with most input tokens served from prompt cache in both conditions (Appendix H). Hermes exposes total-token summaries for a subset of runs and decreases from 483K to 406K median tokens under human skills; Claude Code exposes total-token metadata but not the same fresh/cache split.

Cost Distributions Figure 8 shows the full per-run latency and token distributions behind the medians in Table 8 and Appendix Table 14. Hermes and Claude Code have the lowest median latency, while Codex has the largest latency tail. MUSE-Autoskill uses more tokens than Codex and Claude Code because its loop carries the skill lifecycle and memory machinery, but the generated-skill subset in Table 6 shows that a distilled skill can reduce this cost sharply once it exists.

Generated-Skill Cost Amortization On the 47 tasks where MUSE-Autoskill creates a usable skill, the generated skill is not just more accurate; it is also cheaper to use than both the no-skill and human-skill conditions (Table 6). Accuracy rises from 74.92% without skills and 81.17% with human skills to 85.24% with self-created skills, while median latency drops to 434.7s and median token usage drops to 499K. Creating the skill is a one-time cost: median 363.6K tokens and 156.3s per covered task. Relative to human skills, the generated skill saves about 139K tokens and 321s per reuse, so the token cost breaks even after roughly three reuses and the latency cost after the first reuse.

Bottleneck Analysis MUSE-Autoskill leaves 28 of 75 tasks uncovered in self-creation. These are concentrated in Ops & Planning (12 tasks) and Data Analysis (9), with smaller clusters in Science & Engineering (5) and

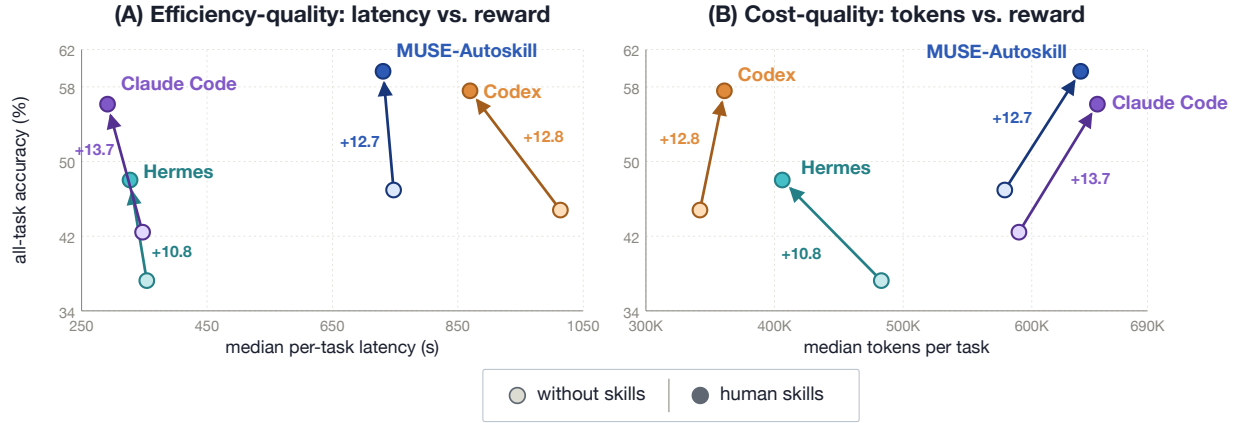


Figure 7 Skill-induced tradeoffs in two dimensions on the 75-task common set. **(A) Latency vs. reward.** Human skills move all four agents upward in reward and reduce median latency. **(B) Tokens vs. reward.** For agents with parseable total-token traces, human skills increase reward with modest token overhead for Codex, Claude Code, and MUSE-Autoskill, while Hermes total tokens decrease on its available summary traces. Colors match Figure 1.

Document Processing (2). This pattern supports the central diagnosis from Table 4: generated skills are strong when a successful source trajectory exists, but Phase 1 exploration still fails to produce a reusable trajectory for many difficult tasks. Future work should therefore focus on improving source-trajectory coverage and on extracting partial diagnostic skills from failed trajectories rather than waiting for a fully successful run.

5 Real-World Deployment and Impact

Beyond the controlled SkillsBench evaluation, the skill-centric design of MUSE-Autoskill is already being adopted in production systems, where skills serve as the common unit of capability shared across agents and users. SkillMarket exposes the skill-creation pipeline to end users, distilling a successful trajectory into a reusable, self-tested skill package without manual authoring; planned releases add skill management and updating, so that deployed skills can be versioned and refined as tasks and environments drift over time. ArkClaw integrates the skill-retrieval component as a find-skill capability, letting an agent locate the most relevant existing skill before synthesizing a new one, and a planned extension treats an entire agent as an invocable sub-agent, so that a single skill can encapsulate delegated multi-agent behavior.

SkillHub operationalizes the full skill lifecycle, covering creation, evaluation, memory, management, and refinement, as a hosted service that gives teams one place to store, evaluate, and govern skills together with their accumulated per-skill experience. Taken together, these deployments show that the lifecycle abstraction is not specific to our benchmark setting: the same retrieve-or-create decision, bundled tests, and per-skill memory carry over to systems built and used by different teams, and an improvement to a shared skill propagates to every agent and product that depends on it.

Looking forward, we expect skills to take on a broader role as the primitive for defining workflows. Rather than hand-wiring agent pipelines, developers will compose and version skills whose bundled tests and memory keep the resulting workflows self-documenting and easier to maintain. This shifts ongoing maintenance cost from bespoke glue code to a shared, continuously evaluated skill ecosystem, and we view these early deployments as evidence that a unified skill lifecycle is a practical foundation for agents whose capabilities compound, rather than erode, as they are maintained at scale.

6 Conclusion

We present a skill-centric agent framework that improves task-solving by acquiring, reusing, and refining skills through a unified lifecycle. By packaging reusable procedures as structured skills, the agent can avoid

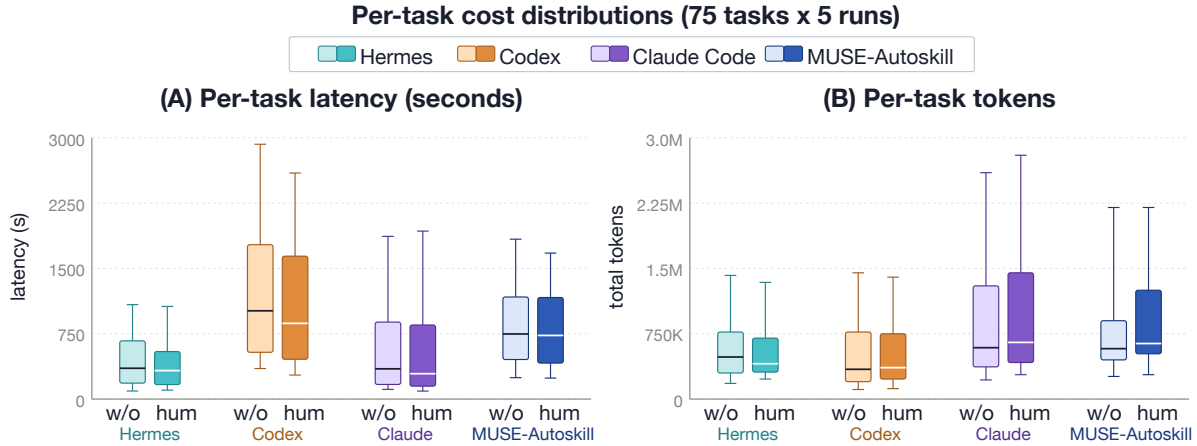


Figure 8 Per-task cost distributions on the 75-task common set. **(A)** Agent execution latency in seconds for Hermes, Codex, Claude Code, and MUSE-Autoskill. **(B)** Total tokens for Hermes, Codex, Claude Code, and MUSE-Autoskill, where parseable token traces are available. Boxes span the IQR, the center line marks the median, and whiskers extend to p10 and p90 for each condition.

rediscovering the same commands, file formats, and validation steps on later runs. MUSE-Autoskill integrates skill creation, evaluation, execution, memory, and management around minimal built-in skills such as *skill_create* and *web_search*. On SkillsBench, it achieves the strongest human-skill accuracy (59.67%), the strongest all-task self-created-skill result (53.42%), and the best transfer into Hermes (51.90%). On SkillLearnBench, it again leads the compared self-creating agents on 100 verified instances. Together with production deployments of skill creation, discovery, and lifecycle management, these results support skill packages as a practical unit for accumulating and reusing agent experience across repeated tasks.

Limitations

Our evaluation covers the 75 SkillsBench tasks that all four compared runtimes can run locally, rather than the full 94-task benchmark; the excluded tasks often have more complex Docker environments and may be harder. Self-created skill coverage is incomplete: MUSE-Autoskill and Codex produce usable skills for 47 of 75 tasks, while Claude Code covers 44. Each covered-task skill is distilled from a single source trajectory, transfer is evaluated only into Hermes, and with 5 runs per task, individual-task confidence intervals remain wide. Full pairwise transfer among all four agents also remains untested.

A further concern is that each self-created skill is generated from one successful Phase 1 trajectory and re-evaluated on the same task. Although the verifier is deterministic and no task-specific ground truth is fed into the skill (Section 4), this protocol may overstate within-task gains. SkillLearnBench uses a separate 100-instance, one-run-per-instance protocol and should be read as corroborating evidence rather than pooled with SkillsBench. Token traces are incomplete across runtimes, so token-cost results are diagnostic. Broader benchmarks are needed before treating the gains as domain-general.

A remaining risk is hallucination in generated skills or in the agent reasoning that invokes them. Skills can encode unsupported facts, brittle file-path or API assumptions, or heuristics overfit to a source trajectory. Unit tests, sandbox execution, verifier feedback, and leakage checks reduce but do not eliminate this risk, so high-impact deployments need stronger provenance tracking, adversarial tests, and human review before skills are shared across teams or exposed to user-facing workflows.

AI tools were used for drafting assistance, wording suggestions, and revision support. The research idea, system prototype, experimental design, experiment execution, result analysis, and final paper-level decisions were led by the human authors, who reviewed the AI-assisted text and take responsibility for the paper’s content, claims, and conclusions.

References

- [1] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- [2] Anthropic. Agent Skills: Equipping Agents for the Real World. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Open standard released December 2025; <https://github.com/anthropics/skills>.
- [3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations, ICLR*, Vienna, Austria, 2024.
- [4] Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. Skillret: A large-scale benchmark for skill retrieval in llm agents. *arXiv preprint arXiv:2605.05726*, 2026.
- [5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for A multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024*, Vienna, Austria, May 7-11, 2024, 2024.
- [6] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of LLM agents: A survey. *CoRR*, abs/2402.02716, 2024.
- [7] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL*, pages 1658–1677, Bangkok, Thailand, 2024.
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR*, Vienna, Austria, 2024.
- [9] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [10] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Shijian Wang, Guanting Dong, Jiajie Jin, Hao Wang, Yinuo Wang, Ji-Rong Wen, Yuan Lu, et al. Omnigaia: Towards native omni-modal ai agents. *CoRR*, abs/2602.22897, 2026.
- [11] Huawei Lin, Yunzhi Shi, Tong Geng, Weijie Zhao, Wei Wang, and Ravender Pal Singh. Agent-omni: Test-time multimodal reasoning via model coordination for understanding anything. *CoRR*, abs/2511.02834, 2025.
- [12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Trans. Assoc. Comput. Linguistics*, 12:157–173, 2024.
- [13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations, ICLR*, Vienna, Austria, 2024.
- [14] Jianglin Lu, Hailing Wang, Xu Ma, Qihua Dong, Mingyuan Zhang, Yizhou Wang, and Yun Fu. Muse: A unified agentic harness for mllms. *arXiv preprint arXiv:2606.03005*, 2026.
- [15] Yuchen Ma, Yue Huang, Han Bao, Haomin Zhuang, Swadheen Shukla, Michel Galley, Xiangliang Zhang, and Stefan Feuerriegel. Skillgen: Verified inference-time agent skill synthesis. *arXiv preprint arXiv:2605.10999*, 2026.
- [16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems, NeurIPS*, New Orleans, LA, 2023.
- [17] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In *The Twelfth International Conference on Learning Representations, ICLR*, Vienna, Austria, 2024.

- [18] Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, et al. Skillos: Learning skill curation for self-evolving agents. [arXiv preprint arXiv:2605.06614](#), 2026.
- [19] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. [CoRR](#), abs/2310.08560, 2023.
- [20] Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In [Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 3](#), pages 2:1–2:22, San Francisco, CA, 2023.
- [21] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In [Advances in Neural Information Processing Systems, NeurIPS](#), Vancouver, BC, Canada, 2024.
- [22] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In [Advances in Neural Information Processing Systems, NeurIPS](#), New Orleans, LA, 2023.
- [23] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. In [Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025](#), pages 5977–6043, 2025.
- [24] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving AI tasks with chatgpt and its friends in huggingface. [CoRR](#), abs/2303.17580, 2023.
- [25] Yaorui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi Gu, Xunliang Cai, Xiang Wang, and An Zhang. Skill1: Unified evolution of skill-augmented agents via reinforcement learning. [arXiv preprint arXiv:2605.06130](#), 2026.
- [26] Yuchen Shi, Yuzheng Cai, Siqi Cai, Zihan Xu, Lichao Chen, Yulei Qin, Zhijian Zhou, Xiang Fei, Chaofan Qiu, Xiaoyu Tan, et al. Youtu-agent: Scaling agent productivity with automated generation and hybrid policy optimization. [arXiv preprint arXiv:2512.24615](#), 2025.
- [27] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In [Advances in Neural Information Processing Systems, NeurIPS](#), New Orleans, LA, 2023.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. [Trans. Mach. Learn. Res.](#), 2024, 2024.
- [29] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. Openhands: An open platform for AI software developers as generalist agents. In [The Thirteenth International Conference on Learning Representations, ICLR](#), Singapore, 2025.
- [30] Ziting Wang, Shize Zhang, Haitao Yuan, Jinwei Zhu, Wei Dong, and Gao Cong. Fdabench: A benchmark for data agents on analytical queries over heterogeneous data. [arXiv preprint arXiv:2509.02473](#), 2025.
- [31] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. [CoRR](#), abs/2308.08155, 2023.
- [32] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In [The Twelfth International Conference on Learning Representations, ICLR](#), Vienna, Austria, 2024.
- [33] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. [arXiv preprint arXiv:2602.12430](#), 2026.
- [34] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In [Advances in Neural Information Processing Systems, NeurIPS](#), Vancouver, BC, Canada, 2024.

- [35] Min Yang, Jinghua Piao, Xu Xia, Xiaochong Lan, Jiaju Chen, Yongshun Gong, and Yong Li. Skillmaster: Toward autonomous skill mastery in llm agents. [arXiv preprint arXiv:2605.08693](#), 2026.
- [36] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. [arXiv preprint arXiv:2603.01145](#), 2026.
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations, ICLR, Kigali, Rwanda, 2023.
- [38] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, pages 13643–13658, 2024.
- [39] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI, pages 19632–19642, Vancouver, Canada, 2024.
- [40] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners. [arXiv preprint arXiv:2505.11942](#), 2025.
- [41] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo FR Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks. [arXiv preprint arXiv:2604.20087](#), 2026.

A Selected Task List

Table 9 lists the 75 SkillsBench tasks used for the four-agent comparison in Figure 1. We keep the original SkillsBench category where available, falling back to the primary task tag when the category field is absent, and group tasks into the four super-domains used in our analysis: 18 Science & Engineering, 18 Data Analysis, 14 Document Processing, and 25 Ops & Planning tasks.

Table 9 The 75-task SkillsBench common set used for the four-agent comparison.

# Task ID	SkillsBench Category	Super-domain
1 3d-scan-calc	engineering	Science & Engineering
2 ada-bathroom-plan-repair	architecture	Document Processing
3 adaptive-cruise-control	control-systems	Science & Engineering
4 azure-bgp-oscillation-route-leak	bgp-route	Ops & Planning
5 bike-rebalance	transportation-logistics	Ops & Planning
6 citation-check	research	Document Processing
7 civ6-adjacency-optimizer	games	Ops & Planning
8 court-form-filling	document-processing	Document Processing
9 crystallographic-wyckoff-position-analysis	materials_science	Science & Engineering
10 dapt-intrusion-detection	security	Ops & Planning
11 data-to-d3	Data Visualization	Data Analysis
12 dialogue-parser	game	Data Analysis
13 dynamic-object-aware-egomotion	video-analysis	Science & Engineering
14 earthquake-plate-calculation	geophysics	Science & Engineering
15 econ-detrending-correlation	economics	Data Analysis
16 edit-pdf	pdf	Document Processing
17 energy-market-pricing	energy	Ops & Planning
18 energy-unit-commitment	energy	Ops & Planning
19 enterprise-information-search	enterprise-search	Ops & Planning
20 exam-block-sequencing	scheduling	Ops & Planning
21 exceltable-in-ppt	Office Operation	Document Processing
22 exoplanet-detection-period	astronomy	Science & Engineering
23 financial-modeling-qa	financial modeling	Data Analysis
24 find-topk-similar-chemicals	chemistry	Science & Engineering
25 fix-druid-loop-hole-cve	Security	Ops & Planning
26 fix-erlang-ssh-cve	erlang bugfix	Ops & Planning
27 flink-query	flink	Ops & Planning
28 flood-risk-analysis	data-processing	Science & Engineering
29 gravitational-wave-detection	astronomy	Science & Engineering
30 grid-dispatch-operator	energy	Ops & Planning
31 hvac-control	control-systems	Science & Engineering
32 invoice-fraud-detection	data-validation	Data Analysis
33 jax-computing-basics	research	Science & Engineering
34 jpg-ocr-stat	data statistics	Document Processing
35 lab-unit-harmonization	healthcare	Data Analysis
36 lake-warming-attribution	data-processing	Science & Engineering
37 latex-formula-extraction	latex-extraction	Document Processing
38 lean4-proof	formal method	Science & Engineering
39 llm-prefix-cache-replay	ml-systems	Ops & Planning
40 manufacturing-codebook-normalization	manufacturing	Ops & Planning
41 manufacturing-equipment-maintenance	manufacturing	Ops & Planning
42 manufacturing-fjsp-optimization	manufacturing	Ops & Planning
43 mars-clouds-clustering	data-science	Science & Engineering
44 multilingual-video-dubbing	multimodal-video-dubbing	Document Processing
45 offer-letter-generator	document-generation	Document Processing
46 organize-messy-files	file-management	Ops & Planning

Continued on next page

# Task ID	SkillsBench Category	Super-domain
47 paper-anonymizer	document-editing	Document Processing
48 parallel-tfidf-search	Parallelization	Data Analysis
49 pddl-tpp-planning	research	Ops & Planning
50 pdf-excel-diff	data-comparison	Document Processing
51 pedestrian-traffic-counting	pedestrian traffic counting	Science & Engineering
52 powerlifting-coef-calc	data-analysis	Data Analysis
53 pptx-reference-formatting	office-suite	Document Processing
54 protein-expression-analysis	data-analysis	Science & Engineering
55 r2r-mpc-control	control-systems	Science & Engineering
56 radar-vital-signs	signal-processing	Science & Engineering
57 react-performance-debugging	web-performance	Ops & Planning
58 reserves-at-risk-calc	financial-analysis	Data Analysis
59 sales-pivot-analysis	data-analysis	Data Analysis
60 sec-financial-report	finance	Data Analysis
61 shock-analysis-demand	financial-analysis	Data Analysis
62 shock-analysis-supply	financial-analysis	Data Analysis
63 simpo-code-reproduction	code reproduction	Ops & Planning
64 software-dependency-audit	security	Ops & Planning
65 spring-boot-jakarta-migration	Legacy Systems	Ops & Planning
66 syzkaller-ppdev-syzlang	security	Ops & Planning
67 taxonomy-tree-merge	ML/NLP	Data Analysis
68 threejs-structure-parser	3d-graphics	Data Analysis
69 threejs-to-obj	3d-graphics	Data Analysis
70 tictoc-unnecessary-abort-detection	database-systems	Ops & Planning
71 travel-planning	travel-planning	Ops & Planning
72 video-silence-remover	media-processing	Document Processing
73 video-tutorial-indexer	multimodal-processing	Document Processing
74 weighted-gdp-calc	financial-analysis	Data Analysis
75 xlsx-recover-data	spreadsheet	Data Analysis

B FDABench Supplementary Evaluation

FDABench-Full [30] evaluates agents on heterogeneous data-analysis tasks. It contains three task families: single-choice questions, where the agent selects one option; multiple-choice questions, where the agent returns a set of options under strict exact-match grading; and report tasks, where the agent produces an analytical report from database-backed evidence. The full public benchmark contains 579 single-choice, 760 multiple-choice, and 668 report task instances. The underlying data instances span Spider2-lite, BIRD, DABStep, and Spider1-style databases, with easy, medium, and hard difficulty labels.

We include FDABench as a supplementary stress test rather than as a headline comparison. The MUSE-Autoskill score values correspond to the public FDABench data-agent leaderboard row listed as Data Analysis Agent (ByteDance Lark Base & Hydra & NovaBase Team). For choice tasks, FDABench reports execution accuracy (EX); MUSE-Autoskill obtains 74.10% EX on single-choice tasks with 281.4s average latency and 179.6M aggregate tokens (310.2K per task), and 49.70% EX on multiple-choice tasks with 183.2s average latency and 166.7M aggregate tokens (219.4K per task). For report tasks, FDABench reports rubric score (RS); MUSE-Autoskill obtains 85.20% RS with 96.6s average latency.

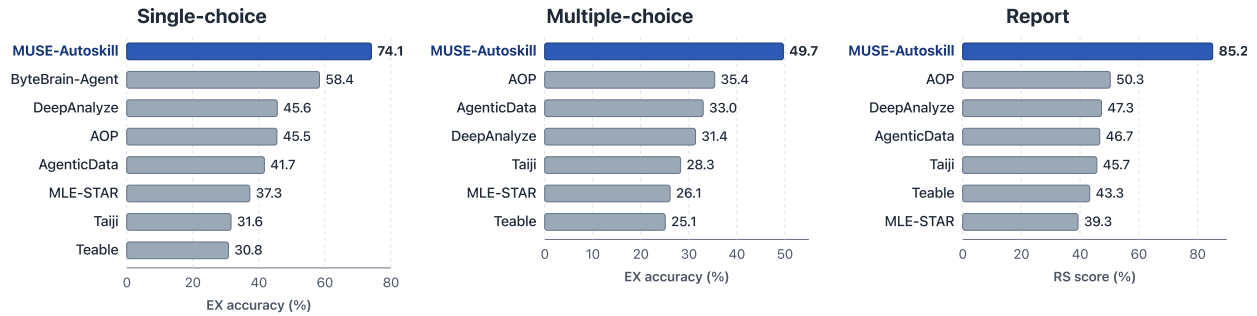


Figure 9 FDABench public data-agent leaderboard comparison for single-choice, multiple-choice, and report tasks. The highlighted bars show MUSE-Autoskill; the score values correspond to the FDABench public row listed as *Data Analysis Agent (ByteDance Lark Base & Hydra & NovaBase Team)*. Values are from the FDABench website’s public `method_aggregated.csv` file at https://fdabench.github.io/static/data/method_aggregated.csv.

C Skill Package Schema

A skill is a self-contained directory rooted at the kebab-case skill name. The directory always contains a top-level `SKILL.md` written in Markdown with a YAML frontmatter block, and may optionally contain the subdirectories `scripts/`, `tests/`, `resources/`, and `references/`. Skills that do not need code consist of `SKILL.md` alone, which is the dominant pattern in practice. The skill identifier is taken from the directory name, and the same name must appear in the frontmatter `name` field; this redundancy lets a skill be moved or copied without breaking its identity. The schema deliberately mirrors Anthropic’s Agent Skills format [2] so that skills produced by MUSE-Autoskill can be loaded by any agent that already understands that format, without translation. The minimum viable skill file is:

```
---
name: <kebab-case skill identifier; must match the directory name>
description: <one-paragraph natural-language description; this is what the
              agent reads when deciding whether to invoke the skill>
---

# <Skill title in Title Case>

## When to use
- Bullet list of triggering task types.

## Core principles
1. Numbered list of invariants the implementation must preserve.

## Recommended tools and libraries
- Concrete library names, CLI commands, or sandbox tools.

## Workflow
Step-by-step procedure the agent should follow at runtime.
```

Catalog routing. The frontmatter `description` field is the only piece of the skill that is surfaced eagerly: at the start of every task the runtime injects a YAML catalog of all available skills (each entry containing just `name` and `description`) into the agent’s system prompt. The body of `SKILL.md` is loaded only after the agent decides, via the `read_skill` tool, that the skill is worth pulling into context. This two-stage lookup keeps the per-call input cost flat in the size of the skill bank: a bank with 100 skills adds only ~5–10K tokens of catalog, not the ~500K tokens that loading every skill body would require.

Subdirectory conventions. The optional subdirectories follow strict per-name conventions, so the agent can rely on layout when reading the skill at runtime: `scripts/` holds executable code (Python, shell, Node) that the skill instructs the agent to run inside the sandbox; `tests/` is used for pytest-compatible validation of code-backed skills and is absent for most text-only procedural skills; `resources/` holds passive auxiliary files (data tables, prompt fragments, reference documents) that the skill loads on demand at execution time; and `references/` (used by some human SkillsBench skills) holds reference documentation that the agent may read but is not expected to execute. When `tests/` is present, failed tests block registration; otherwise registration relies on the available sandbox/runtime checks described above. A skill never embeds dependencies: it relies on the sandbox image (or runtime `pip install` via the terminal tool) for any packages it needs, which keeps the skill bank itself a pure-text artifact safe to version-control and ship as a tarball.

Skill-level memory. Alongside the on-disk skill, each skill gets a sibling `.memory.md` file (created lazily on first write) into which the agent appends notes, lessons, and usage observations across tasks. This file is the concrete realisation of the skill-level memory described in Section 3.2. It is intentionally outside the skill directory’s published surface area (the leading dot, and exclusion from any `.tar` the user might ship) so that transferring the skill does not transfer experience accumulated from prior runs; experience is per-agent.

D File-System Layout

This appendix documents the on-disk layout the agent assumes at runtime. Every path is configurable; we list the defaults so that an outside reader can understand where each component of a published trajectory came from in the run archive.

Agent home directory. On the host the agent runtime defaults to `$HOME/.autoskill` (overridable via the `AUTOSKILL_HOME` environment variable). It is created on first launch and contains all persistent state that is not part of an individual session:

```
~/autoskill/
+-- skills/                                # the skill bank: one directory per skill
|   +-- pdf-form-update-redaction/
|   |   +-- SKILL.md                      # frontmatter + body
|   |   +-- .memory.md                   # per-skill memory (Section 3.4)
|   |   +-- scripts/                     # optional: executable code
|   |   +-- tests/                       # optional: pytest-compatible
|   |   +-- resources/                   # optional: data / docs
|   +-- csv-summarize/
|   |   +-- SKILL.md                      # the typical "doc-only" skill
|   +-- ...
+-- memory/
|   +-- long_term_memory/
|   |   +-- memory.md                    # cross-session notes, lessons learned
+-- sessions/                             # per-session workspaces (see below)
    +-- 2d9b1c67f73947c4863b26a45c5098a8/
    +-- ...
```

Per-session workspace. A new directory under `$AUTOSKILL_HOME/sessions/<session_id>/` is created for each task invocation. The session id is a UUID-like string that also names the directory; the runtime persists the agent’s complete state into this directory at the end of every session. Inside one session:

```
sessions/<session_id>/
+-- instruction.md                        # the task prompt the agent received
+-- submitted_inputs/                    # files supplied by the caller
+-- submitted_skillhub/                  # any injected skills supplied at task start
+-- result_output_files/                 # final artifacts the agent produced
```

```

+-- agent_message.md           # final-answer text returned to the caller
+-- agent_stdout.txt           # log stream incl. per-call token usage
+-- events.jsonl               # one JSON event per tool call / turn
+-- memory.md                  # short-term (session-scoped) memory
+-- ctx_state.json              # serialised AgentContext (for resume)
+-- profile.json                # latency breakdown (setup, exec, verifier)
+-- run_meta.json               # reward, turn count, model, ...

```

The most important files for reproducibility are `events.jsonl`, which contains a strictly-ordered stream of every plan / action / observation in the run (we use it for Appendix J), and `ctx_state.json`, the snapshot used by the cross-session resume mechanism (Section 3.4). `ctx_state.json` contains the full conversation DAG: every `ConversationNode` with its original input, the `compressed_input` (if Level-1 compression was applied), and both pointer sets (`parent_id` for the active chain, `history_prev/history_next` for the original ordering).

Sandbox layout. Each invocation of `create_sandbox` spawns an isolated process with its own filesystem rooted at `/sandbox` (the exact backing depends on the sandbox factory: local processes, Docker containers, or a managed sandbox service all expose the same interface). Files the agent uploads via `sandbox_upload` land under `/sandbox/inputs/`; files produced by scripts go under `/sandbox/outputs/` and are pulled back with `sandbox_download` when the agent needs them. The sandbox is destroyed at the end of the session (or earlier if the agent explicitly calls `close_sandbox`), so no skill execution can affect host state.

Memory file format. All three memory files (long-term, short-term, and per-skill) share the same plain-Markdown format: an append-only writer appends a single block of the form

```

## 2026-05-07 10:34:33 UTC
<agent-written content, one short paragraph or list>

```

to the appropriate file. Read access is line-buffered and unparsed; the agent never edits or deletes existing entries, which keeps memory append-only and makes the file safe to read from multiple sessions concurrently.

E Hyperparameters and Runtime Configuration

Table 10 lists every runtime constant used in the SkillsBench experiments. All values were held fixed across the 75-task common set; we did not perform per-task tuning. The values fall into four groups, each with a specific design intent and evaluation role.

Compression thresholds. `COMPRESS_TOKEN_THRESHOLD` (180K) is set just below the model’s hard 200K context limit, leaving a $\sim 10\%$ headroom so a Level-2 compression call can itself fit in context. `NODE_COMPRESS_TOKEN_THRESHOLD` (15K) is the size at which a single tool output stops being amortisable across turns and starts dominating the prompt cache cost; below this threshold, leaving the original verbatim is preferable to summarising. `COMPRESS_KEEP_FIRST_TURNS` and `COMPRESS_KEEP_LAST_TURNS` (both 5) ensure that the task framing (the system prompt and the first few turns of grounding) and the immediate working context (the most recent five turns) are always sent verbatim; only the middle of the conversation is eligible for compression. In practice 5+5 turns are sufficient overhead even on the longest tasks we observed (max 69 turns).

Tool execution timeouts. The hierarchy `TOOL_TIMEOUT_SECONDS` (300) > `VERIFY_COMPLETION_TIMEOUT_SECONDS` (120) > `TERMINAL_TIMEOUT_SECONDS` (60) = `EXEC_CODE_TIMEOUT_SECONDS` (60) reflects the expected wall-clock cost of each operation: a generic tool (e.g. a multi-step skill invocation) may block for several minutes, the completion checker is bounded to a single LLM call plus diagnostics, and individual shell / Python snippets are kept short to keep the ReAct loop responsive. `MODEL_TIMEOUT_SECONDS` (300) is a guard against API hangs; on success the actual LLM call completes in 5–30 s. `TOOL_TEXT_LIMIT` (8,192 characters) is the hard cap on a single tool output before truncation, which protects the active chain from a single misbehaving tool dumping an entire log file.

Retry and verification. `MAX_RETRY` (5) is the per-call exponential-backoff budget for transient API failures (HTTP 429, 5xx). `VERIFY_COMPLETION_TURN_THRESHOLD` (4) is the smallest number of turns after which the agent is allowed to call `final_answer`; below this threshold a `verify_completion` pre-check is forced, which prevents the agent from prematurely terminating on tasks it has barely engaged with.

Backbone and agents. All four agents share the same model deployment, `gpt-5.5-2026-04-24` (paper label: GPT-5.5 (04/24/2026)). We did not set temperature, top- p , or other decoding overrides, so provider defaults are used throughout. The evaluated systems are **Hermes**, **Codex**, **Claude Code**, and **MUSE-Autoskill** (this work, running its own backend as described in Section 4). Claude Code’s model calls are routed to `gpt-5.5-2026-04-24` through a compatibility bridge. Accuracy differences should therefore be interpreted as differences in agent prompts, tool definitions, compression policies, context handling, and skill-loading behaviour rather than differences in the model backbone. Every task is run 5 times in independent Docker containers; the SkillsBench harness controls per-task wall-clock budget.

Table 10 All runtime constants used in the experiments. The same model-level settings were used for Hermes, Codex, Claude Code, and MUSE-Autoskill. Hermes, Codex, and Claude Code inherit only model-level constants; their tool-execution and compression behaviour is governed by their own agent systems. Tasks were graded by the SkillsBench verifier in unmodified Docker environments.

Parameter	Value	Role
<i>Backbone model</i>		
paper model label	GPT-5.5 (04/24/2026)	shared across all four agents
deployment name	<code>gpt-5.5-2026-04-24</code>	shared model backend
temperature	default	no sampling override
top- p	default	no sampling override
<i>Context compression (see Appendix G)</i>		
<code>COMPRESS_TOKEN_THRESHOLD</code>	180,000	total-context trigger for Level-2
<code>NODE_COMPRESS_TOKEN_THRESHOLD</code>	15,000	per-node trigger for Level-1
<code>COMPRESS_KEEP_FIRST_TURNS</code>	5	oldest turns kept verbatim
<code>COMPRESS_KEEP_LAST_TURNS</code>	5	most recent turns kept verbatim
<i>Tool execution</i>		
<code>TOOL_TEXT_LIMIT</code>	8,192 chars	per-call tool output truncation
<code>TOOL_TIMEOUT_SECONDS</code>	300	generic tool deadline
<code>TERMINAL_TIMEOUT_SECONDS</code>	60	shell-command deadline
<code>EXEC_CODE_TIMEOUT_SECONDS</code>	60	Python-snippet deadline
<code>VERIFY_COMPLETION_TIMEOUT_SECONDS</code>	120	deadline for the completion checker
<code>MODEL_TIMEOUT_SECONDS</code>	300	deadline for a single LLM call
<code>MAX_RETRY</code>	5	per-call retry budget on API failures
<code>VERIFY_COMPLETION_TURN_THRESHOLD</code>	4	turns after which <code>final_answer</code> requires verification
<i>Evaluation protocol</i>		
runs per task	5	independent Docker containers
timeout per task	inherited from SkillsBench harness	varies by task

F SkillLearnBench Memory-System On/Off Ablation

We ran an additional diagnostic ablation on SkillLearnBench to isolate the effect of the MUSE-Autoskill memory system. This experiment is separate from the headline SkillLearnBench table in Section 4. It uses the human-skill setting together with a two-pass verifier-feedback protocol: in round 1 the agent attempts the instance normally, the verifier is run, and in round 2 a fresh environment receives the original task plus the round 1 verifier feedback. The primary verifier-feedback outcome is the round 2 result, and we also report round 1 as a pre-feedback diagnostic. The memory-system-off condition treats every instance independently. The memory-system-on condition executes instances sequentially within each task, restores the task-scoped memory state before each instance, and copies the updated memory state back after the run so that later instances can reuse accumulated observations. The round 2 prompt explicitly forbids saving oracle data, expected answers, concrete final outputs, or other instance-specific answer keys into memory.

We report all 100 runs in each condition. For each condition we report both the first attempt (round 1) and the verifier-feedback rerun (round 2). Accuracy counts verifier successes over the 100 runs in each condition; token, turn, and latency statistics summarize the corresponding runs.

Table 11 SkillLearnBench memory-system on/off ablation under the two-pass verifier-feedback protocol. Each condition contains 100 runs; accuracy counts verifier successes, while token, turn, and latency columns summarize run-level execution cost.

Condition	Round	Runs	Successes	Accuracy	Tokens/run	Turns/run	Latency/run
Memory system off	Round 1	100	48	48/100 (48.0%)	524.4k	12.7	419.1s
Memory system off	Round 2	100	60	60/100 (60.0%)	408.4k	11.6	338.6s
Memory system on	Round 1	100	64	64/100 (64.0%)	467.3k	11.4	347.7s
Memory system on	Round 2	100	74	74/100 (74.0%)	423.9k	11.7	295.4s

In this experiment, the memory system improves the round 2 verifier-feedback accuracy by +14.0 percentage points (60/100 $\rightarrow$ 74/100). The same pattern is already visible before feedback: round 1 accuracy rises from 48/100 (48.0%) to 64/100 (64.0%). The memory system also reduces round 1 execution cost (524.4k $\rightarrow$ 467.3k tokens/run; 12.7 $\rightarrow$ 11.4 turns/run; 419.1s $\rightarrow$ 347.7s). In round 2, tokens are slightly higher with the memory system (408.4k $\rightarrow$ 423.9k), turns are essentially unchanged (11.6 $\rightarrow$ 11.7), and latency is lower (338.6s $\rightarrow$ 295.4s). Overall, the result suggests that the memory system improves reliability on repeated SkillLearnBench tasks without increasing the number of reasoning turns.

G Compression Algorithm

Context compression is invoked by `maybe_compress_history(ctx, model)` at the start of every ReAct turn, immediately after the agent’s response is appended to the conversation and before the next LLM call is issued. The function returns silently when the active chain is under budget (which is the common case at the start of a run) and only triggers an LLM-summarisation call when the total token estimate crosses `COMPRESS_TOKEN_THRESHOLD`. We implement two levels of progressively more aggressive compression; in our SkillsBench runs Level-1 is sufficient for the vast majority of contexts that exceed the budget and Level-2 fires only on the longest-running tasks (turn count >50). Both levels operate exclusively on the active chain (the linked list reachable via `parent_id`); the immutable `history_prev/history_next` pointers are never rewritten, so any prior state can still be reconstructed for cross-session resume or for post-hoc trajectory analysis. The high-level control flow is:

```
def maybe_compress_history(ctx, model):
    chain = walk(parent_id from tip to root)
    total_tokens = sum(estimate_tokens(node) for node in chain)
    if total_tokens <= COMPRESS_TOKEN_THRESHOLD:
        return # under budget; nothing to do

    # ---- Level 1: per-node, in-place summary on oversized nodes ----
    # never touch the first K=5 or last K=5 turns
    middle = chain[KEEP_FIRST : -KEEP_LAST]
    for node in middle:
        if estimate_tokens(node) > NODE_COMPRESS_TOKEN_THRESHOLD:
            summary = model.summarize(node.input + node.model_output)
            node.compressed_input = summary
            node.is_node_compressed = True # reads return summary

    if recompute_total(chain) <= COMPRESS_TOKEN_THRESHOLD:
        return # Level 1 was enough

    # ---- Level 2: collapse the middle span into one summary node ----
    span = chain[KEEP_FIRST : -KEEP_LAST]
    summary = model.summarize(concat(span))
    sNode = new ConversationNode(
        is_summary = True,
        parent_id = chain[KEEP_FIRST - 1].node_id,
        compressed_input = summary,
```



```
)
chain[-KEEP_LAST].parent_id = sNode.node_id # rewire chain
```

Cost. Compression itself costs LLM calls: Level-1 issues at most one summarisation call per oversized node, Level-2 issues exactly one summarisation call per trigger. Because the threshold is much larger than a typical tool output, the amortised cost is small (one extra LLM call every ~ 10 – 20 ReAct turns on the long-running tasks we observed). The summary calls use the same backbone model (GPT-5.5 (04/24/2026)) at provider defaults; we did not separately tune them.

Audit trail. The original `node.input` field is never mutated. Reads through the active-chain reader return `compressed_input` when `is_node_compressed` is `True`, so the active chain shrinks; reads through the full-history reader ignore the `compressed_input` field and walk the immutable history pointers, so the original ordering is recoverable. Level-2’s synthetic summary node has `is_summary=True` and, by construction, no history pointers, so the full-history reader skips it and recovers exactly the original sequence of turns. This means any compressed run can be “replayed” for analysis without re-running the agent.

Why not just truncate? A simpler alternative (drop the oldest middle turns when the budget is exceeded) would lose information silently. Our preliminary experiments showed that on multi-step tasks the agent revisits early-context facts roughly 30–40% of the time (e.g. to recheck an input filename or recall a parsing-format detail); truncation forced wasteful re-discovery. Summarisation preserves these facts at $\sim 1/10$ the token cost, which is the regime where the LLM-call overhead pays for itself.

H Detailed Token Breakdown

Token accounting is diagnostic rather than a headline comparison because fresh/cache token traces are complete for MUSE-Autoskill and mostly complete for Codex, while Hermes and Claude Code expose total-token metadata without the same normalized fresh/cache decomposition in the current logs. Table 12 reports medians over available runs from the 75-task common set where that decomposition is parseable, splitting prompt tokens into the fresh component and the cached component, and reporting output and reasoning tokens separately for each condition.

Two patterns are worth flagging. First, both MUSE-Autoskill and Codex show substantial prompt-cache use: median cached input is larger than median fresh input in every reported condition. Second, human skills increase MUSE-Autoskill’s median token footprint, primarily through additional skill-catalog and skill-body context, while Codex changes only modestly. Since this detailed breakdown excludes total-only Hermes and Claude Code component rows, we use latency rather than token cost for the complete four-agent efficiency comparison in Appendix J.

Table 12 Per-task token usage diagnostic on the 75-task common set. Columns are medians computed independently over runs with parseable token traces, so “Total” need not equal the sum of the displayed median components. “Reasoning” is counted within “output” by the model API.

Agent	Condition	Fresh in	Cached in	Output	Reasoning	Total	<i>n</i>
Codex	without skills	133,211	210,880	7,676	2,246	342,220	359
Codex	human skills	146,031	199,680	7,007	1,987	360,683	361
MUSE-Autoskill	without skills	222,155	322,688	13,314	4,986	579,499	375
MUSE-Autoskill	human skills	266,030	359,424	13,059	4,472	638,000	375

I Per-Domain Accuracy with Standard Deviation

Table 13 reports the per-domain mean accuracy together with task-level standard deviation, for each of the four agents and both skill conditions. Standard deviation is computed across the per-task means (each task’s mean is itself averaged over 5 runs); a high σ signals that the agent’s accuracy varies considerably across

tasks in that domain (some tasks land near 100%, others near 0%) and does not directly reflect run-to-run noise.

Skills improve all four domains for all four agents in aggregate. The largest domain lift is in Ops & Planning for Codex, Claude Code, and MUSE-Autoskill, while Hermes gains most in Science & Engineering. MUSE-Autoskill leads the human-skill column in Science & Engineering, Document Processing, and Ops & Planning; Claude Code leads Data Analysis. High standard deviations remain common because each domain mixes near-solved tasks with tasks that no agent solves reliably.

Table 13 Per-domain accuracy (%) with task-level standard deviation on the 75-task common set. Lift is the difference of means (w/ human skills – w/o skills).

Agent	Domain	<i>n</i> tasks	w/o skills	w/ human skills	Lift
Hermes	Science & Engineering	18	40.61 ± 45.29	58.89 ± 39.71	+18.28
Hermes	Data Analysis	18	36.34 ± 43.30	40.60 ± 40.09	+4.26
Hermes	Document Processing	14	54.29 ± 37.36	58.57 ± 41.03	+4.29
Hermes	Ops & Planning	25	25.92 ± 36.87	39.64 ± 36.31	+13.72
Codex	Science & Engineering	18	54.52 ± 43.14	67.12 ± 40.28	+12.60
Codex	Data Analysis	18	38.31 ± 41.38	50.18 ± 44.48	+11.87
Codex	Document Processing	14	68.57 ± 33.56	72.86 ± 32.61	+4.29
Codex	Ops & Planning	25	29.16 ± 34.41	47.48 ± 41.00	+18.32
Claude Code	Science & Engineering	18	52.33 ± 40.07	63.88 ± 38.72	+11.55
Claude Code	Data Analysis	18	38.05 ± 39.63	52.59 ± 44.05	+14.54
Claude Code	Document Processing	14	62.86 ± 39.90	64.29 ± 37.17	+1.43
Claude Code	Ops & Planning	25	27.00 ± 36.78	48.60 ± 41.82	+21.60
MUSE-Autoskill	Science & Engineering	18	54.57 ± 40.93	67.97 ± 40.99	+13.40
MUSE-Autoskill	Data Analysis	18	39.49 ± 44.36	51.48 ± 47.02	+11.99
MUSE-Autoskill	Document Processing	14	67.14 ± 35.14	74.29 ± 34.99	+7.14
MUSE-Autoskill	Ops & Planning	25	35.52 ± 40.80	51.40 ± 39.78	+15.88

J Latency and Turn-Count Distribution

Table 14 reports the percentile distribution of per-task latency (agent execution time in seconds, excluding the SkillsBench verifier) and ReAct turn counts. Both are computed across 75 tasks × 5 runs = 375 runs per (agent, condition) cell. **Hermes** is the fastest runtime by median latency, followed by Claude Code, MUSE-Autoskill, and Codex. Claude Code has a longer high-percentile tail than Hermes despite a comparable median, while Codex remains the slowest median runtime. MUSE-Autoskill runs deeper loops than Hermes and Claude Code, but its tail is shorter than Codex’s.

A second observation is that human skills reduce median latency for every agent while improving accuracy: Hermes drops from 354.0s to 327.3s, Codex from 1013.6s to 869.5s, Claude Code from 347.3s to 291.4s, and MUSE-Autoskill from 747.6s to 730.8s. Turn counts do not uniformly decrease, so the complete efficiency claim is about wall-clock latency rather than fewer ReAct steps.

Table 14 Distribution of per-task agent latency (seconds) and ReAct turn counts, by agent and skill condition on the 75-task common set. “p10” / “p25” / “p75” / “p90” are the 10th, 25th, 75th, and 90th percentile. “max” for turns is the absolute maximum observed across all runs in that cell.

Agent	Cond.	Latency (s)					Turns				<i>n</i>
		p10	p25	median	p75	p90	p25	median	p75	max	
Hermes	w/o	92.5	183.2	354.0	668.7	1084.2	9	14	19	73	375
Hermes	w/	102.9	166.8	327.3	545.8	1063.8	9	13	18	61	375
Codex	w/o	350.8	537.3	1013.6	1772.8	2926.1	8	12	19	57	375
Codex	w/	275.8	457.1	869.5	1641.0	2596.0	8	14	22	47	375
Claude Code	w/o	111.4	168.1	347.3	884.2	1868.2	12	19	28	227	375
Claude Code	w/	91.7	148.4	291.4	850.8	1929.1	14	22	32	109	375
MUSE-Autoskill	w/o	245.7	454.4	747.6	1172.3	1836.2	15	20	26	72	375
MUSE-Autoskill	w/	241.3	413.5	730.8	1166.3	1676.6	15	20	26	82	375

K Self-Created Skill Stability

Table 15 compares run-to-run stability for self-created skills on each agent’s own covered subset. For each task, we compute the mean reward and standard deviation over the canonical five independent runs, then average these task-level statistics within the covered subset. Lower standard deviation and lower mean absolute deviation indicate that repeated runs produce more consistent rewards, regardless of whether the mean is 3/5, 4/5, or 5/5. MUSE-Autoskill has both the highest covered-task reward and the lowest run-to-run dispersion: its average per-task standard deviation is 0.109, compared with 0.124 for Codex and 0.182 for Claude Code. It also has the fewest non-constant tasks (13/47) and the most low-variance tasks (35/47).

Table 15 Run-to-run stability of self-created skills on each agent’s covered subset. Rewards are computed over the canonical five independent runs per task. “Non-constant” counts tasks whose five rewards are not identical. “Low-var.” counts tasks with per-task reward standard deviation at most 0.1.

Agent	Covered	Avg. Reward	Avg. Std.	Avg. MAD	Non-constant	Low-var.
Codex	47	75.83%	0.124	0.110	17/47	32/47
Claude Code	44	75.45%	0.182	0.160	21/44	26/44
MUSE-Autoskill (Ours)	47	85.24%	0.109	0.096	13/47	35/47

This stability pattern suggests that MUSE-Autoskill-created skills often encode a more executable procedure than general skill descriptions. Human- or agent-authored skills can provide useful domain knowledge while still leaving run-specific choices to the agent. In contrast, a skill distilled from a successful trajectory tends to preserve concrete procedural details, such as command sequences, file paths, output schemas, validation checks, and task-specific failure modes. These details narrow the action space during reuse, making repeated executions more likely to follow the same successful path. The benefit is not universal: trajectory-derived skills can also overfit to brittle source-run assumptions, as illustrated by the `hvac-control` regression case study in Section 4.3 on held-out reruns.

L Skill-Generation Failures: The 28 MUSE-Uncovered Tasks

MUSE-Autoskill produced no usable self-created skill for 28 of the 75 tasks under the strict 75-task protocol. These tasks contribute 0% to the all-task average in Table 4. Their distribution characterises the current limits of inference-time skill synthesis.

The uncovered set is concentrated in Ops & Planning and Data Analysis, with smaller clusters in Science & Engineering and Document Processing. This supports the bottleneck analysis in Section 4.3: self-created skills are strong when a reusable source trajectory exists, but coverage still depends on Phase 1 exploration finding such a trajectory. A natural next direction is to extract partial skills from failed trajectories, capturing diagnostic moves that worked even when the run ultimately ended at reward 0.